ARTIFICIAL INTELLIGENCE

CSE 3201

Uninformed Search

Uninformed Search

- Uninform search is also called Blind search. It is the search which has no information about its domain. The only thing that a blind search can do is to differentiate between a non goal state and a goal state. These methods do not need domain knowledge but they are less efficient in result.
- Uninform strategies don’t use any information about how close a node might be to a goal. They differ in the order that the nodes are expanded. The most important uninformed techniques are breadth first search, depth first search, IDDFS, uniform cost search.

Uninformed Search Strategies

- Use only information available in the problem formulation

  -Breadth-first

  -Depth-first

  -Depth-limited

  -Iterative deepening

  -Uniform Cost

  -Bi-Directional

Breadth-First Search

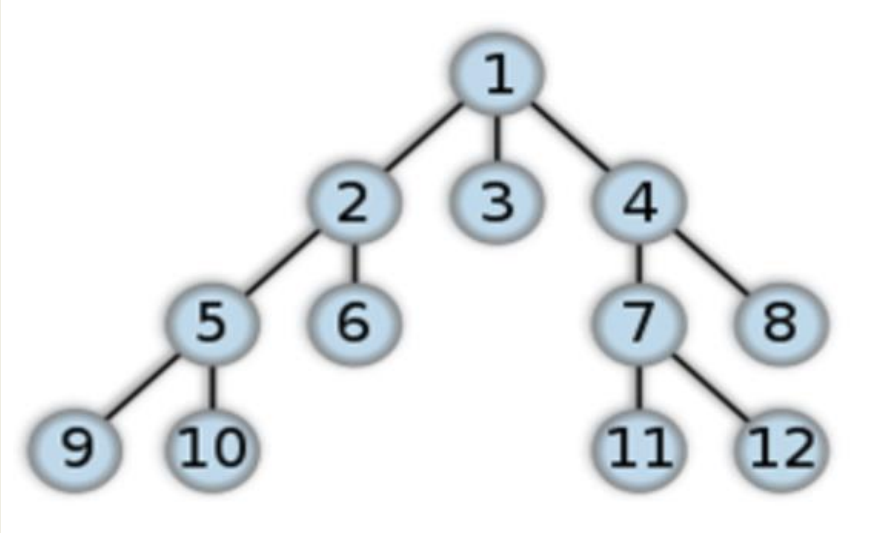
- Expand shallowest unexpanded node
- All the nodes reachable from the current node are explored first

– achieved by the TREE-SEARCH method by appending newly generated nodes at the end of the search queue.

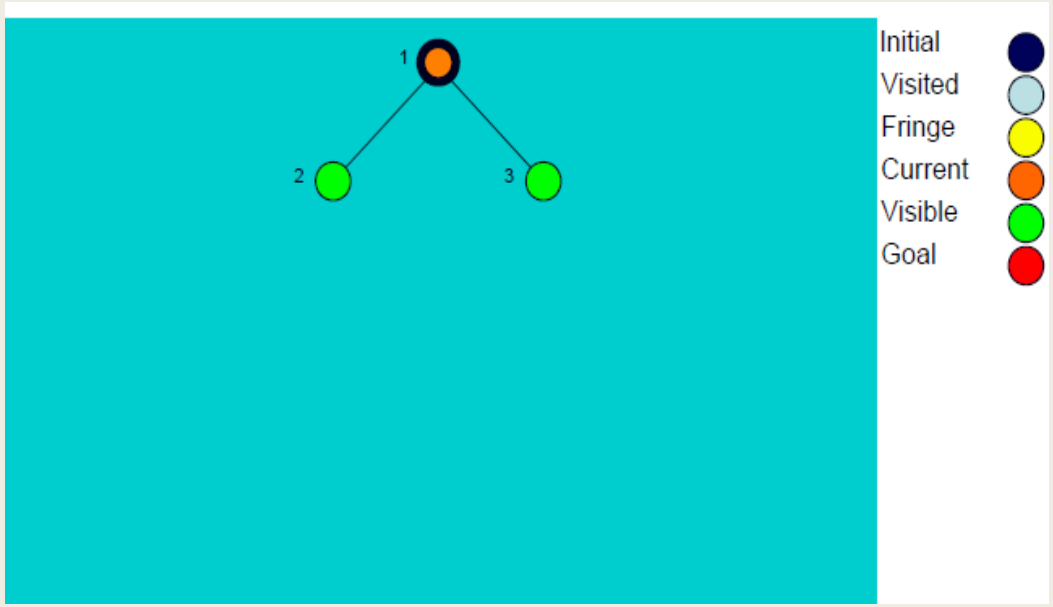
- Implementation:

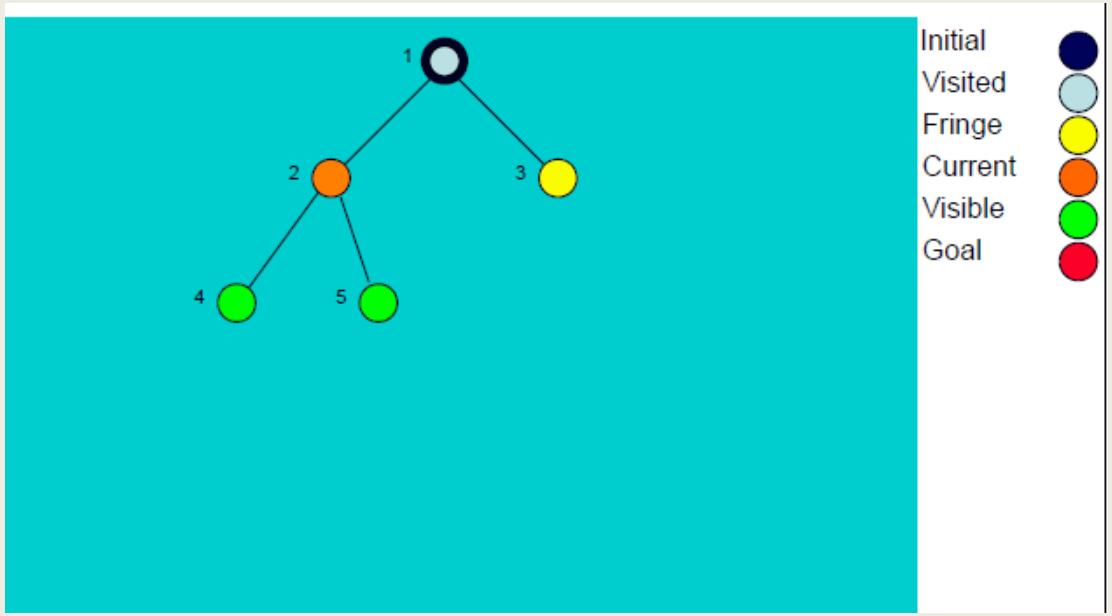
– A **FIFO** queue, i.e., new successors go at end

- Order of expanding nodes:

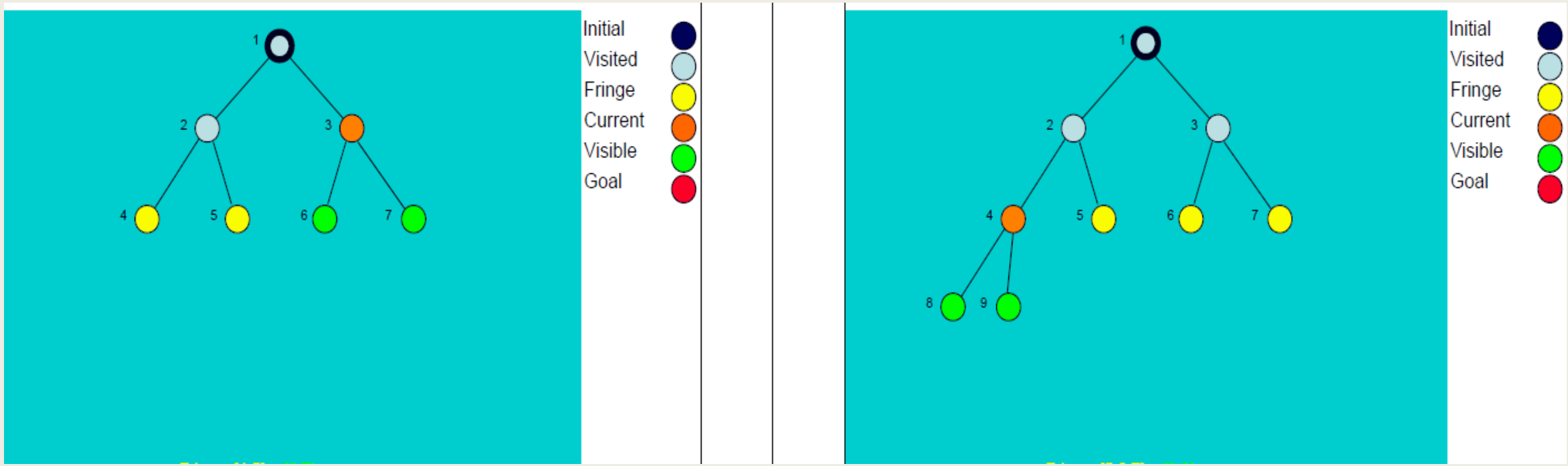


Breadth-First Snapshot

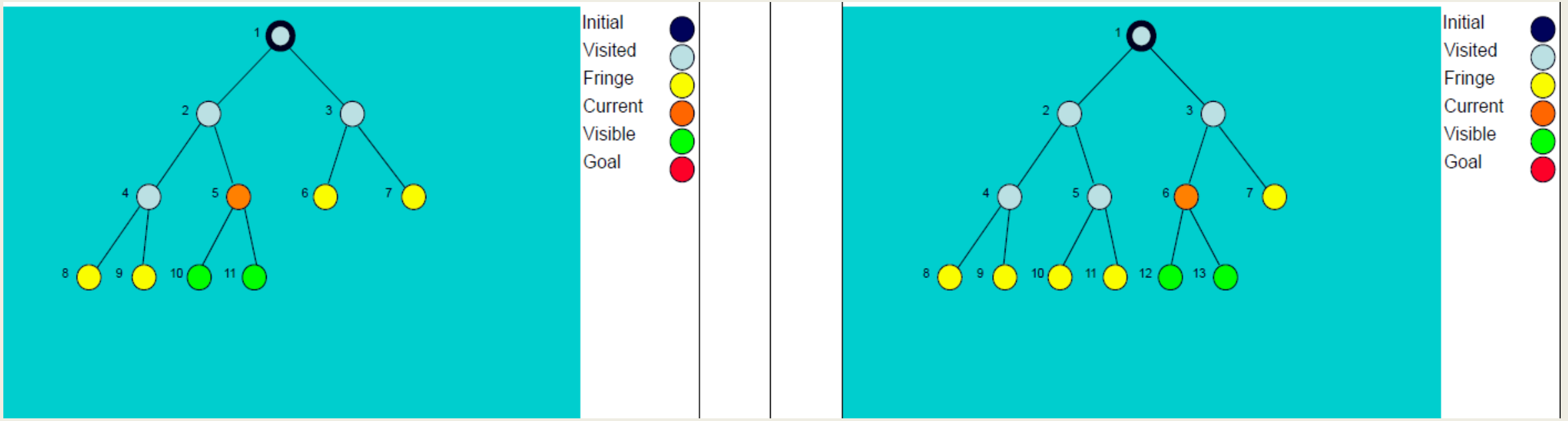




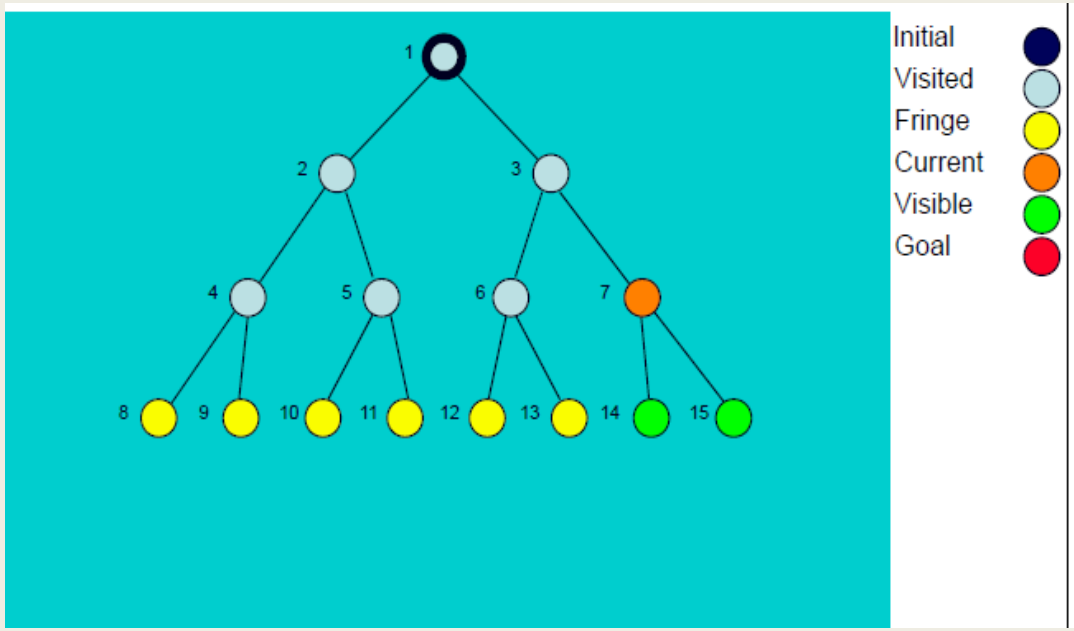
Breadth-First Snapshot

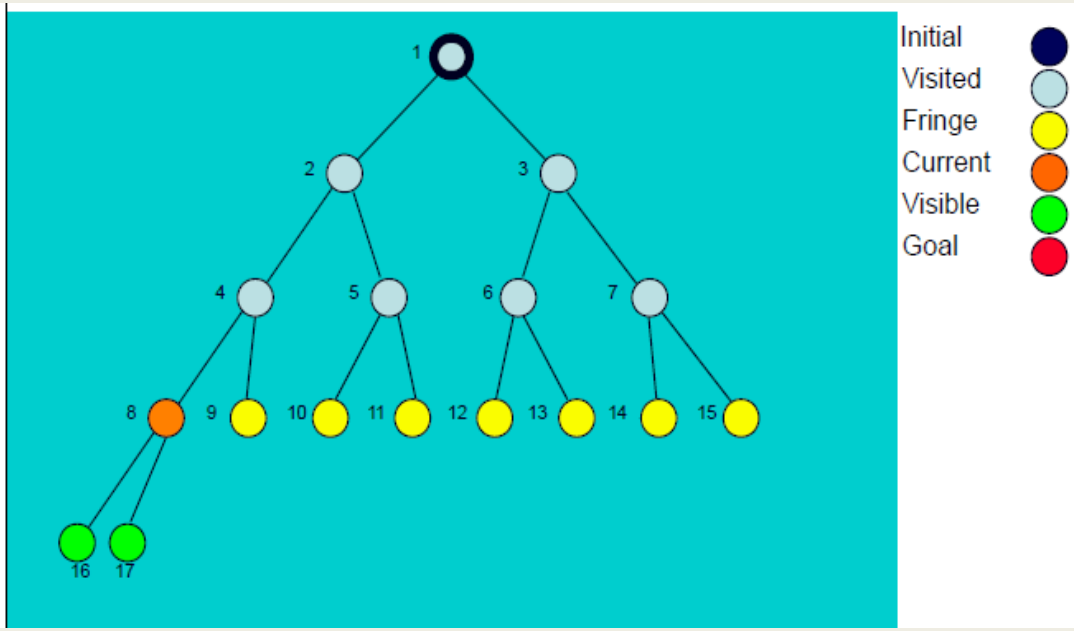


Breadth-First Snapshot

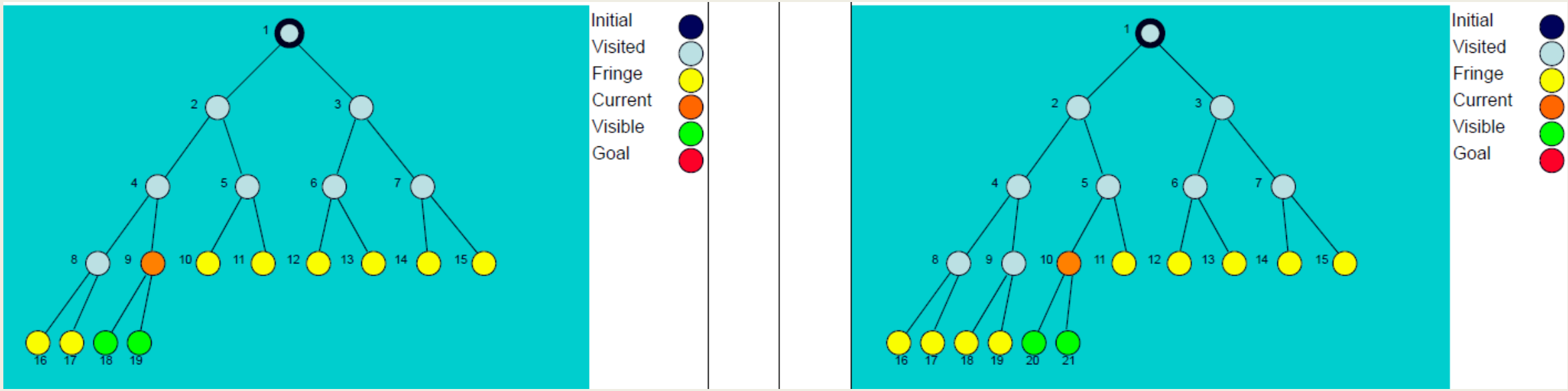


Breadth-First Snapshot

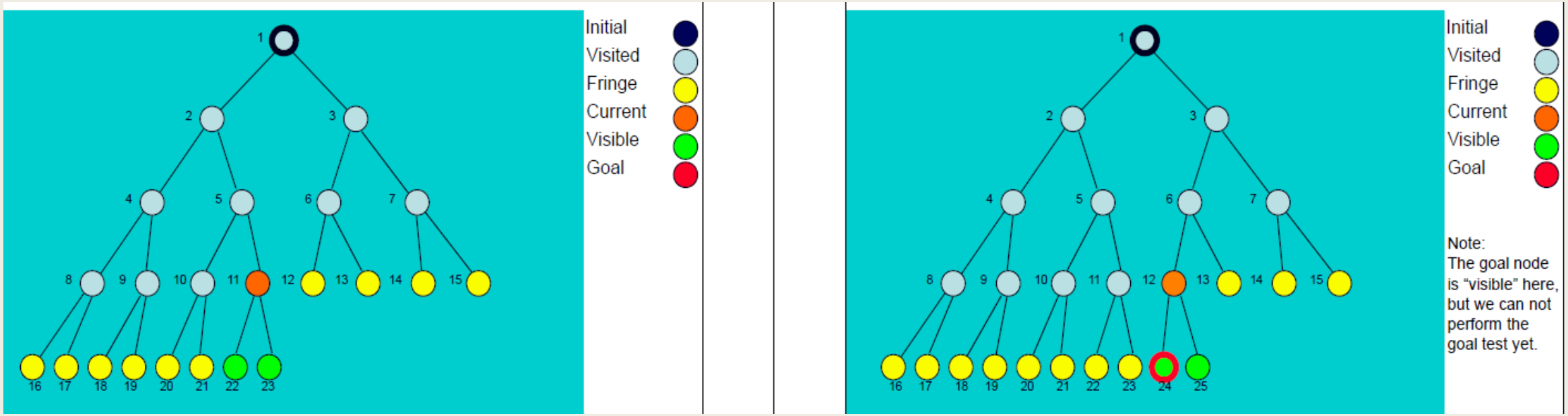




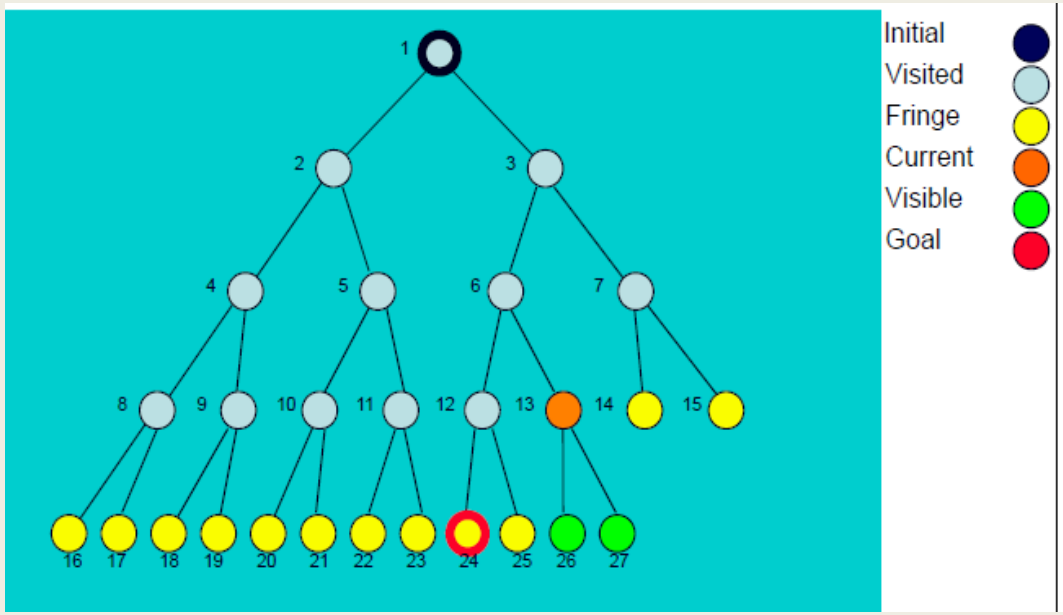
Breadth-First Snapshot

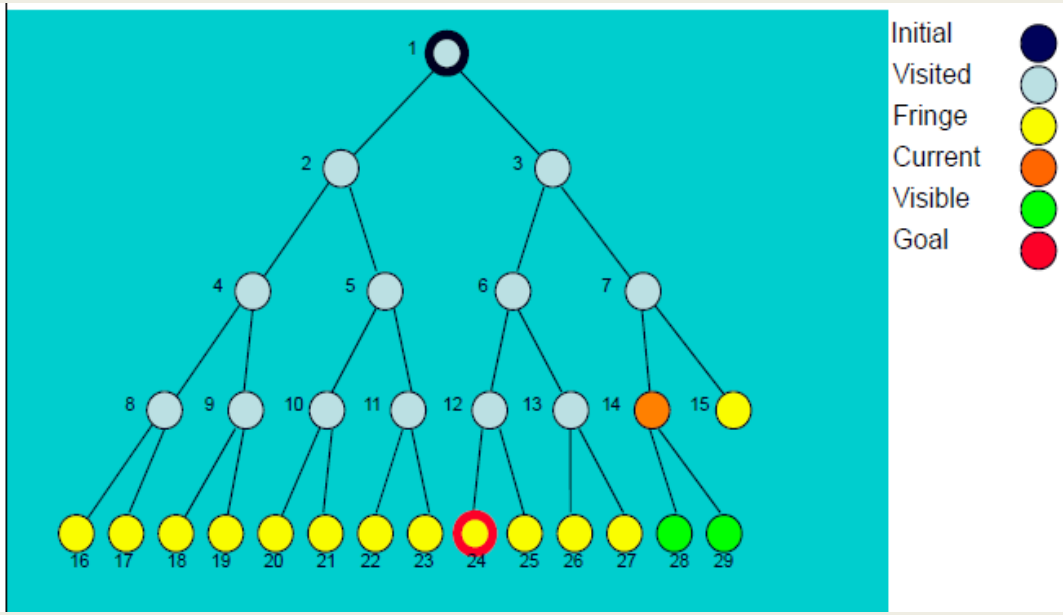


Breadth-First Snapshot

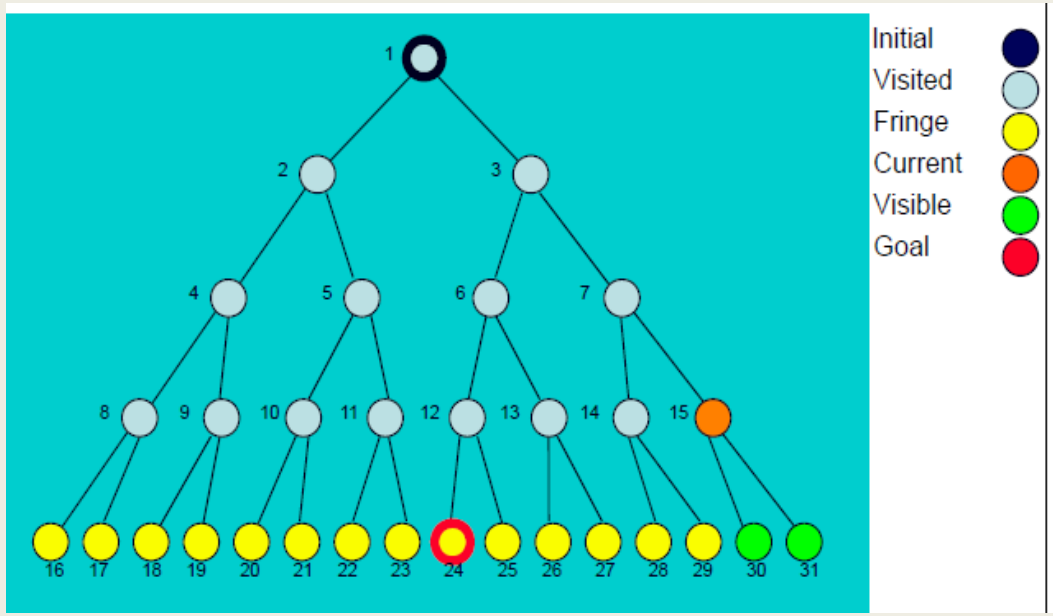


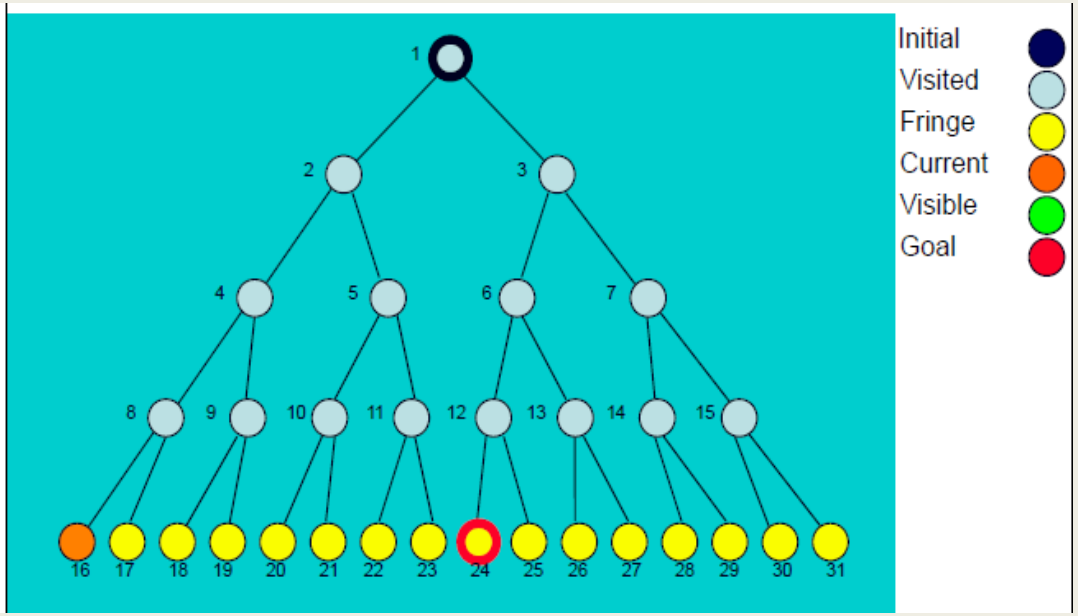
Breadth-First Snapshot



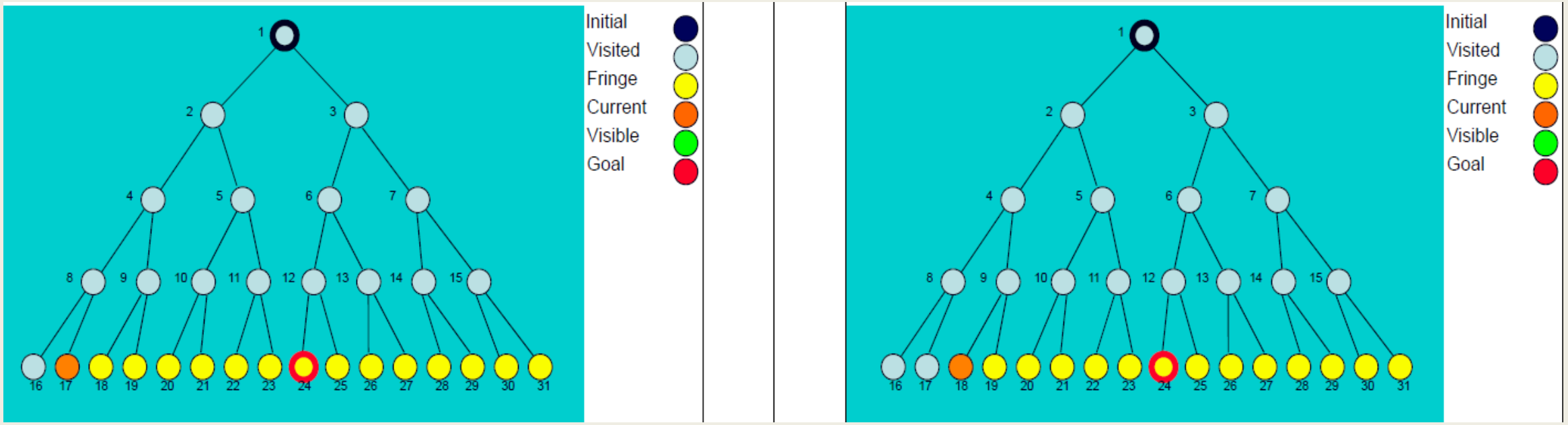


Breadth-First Snapshot

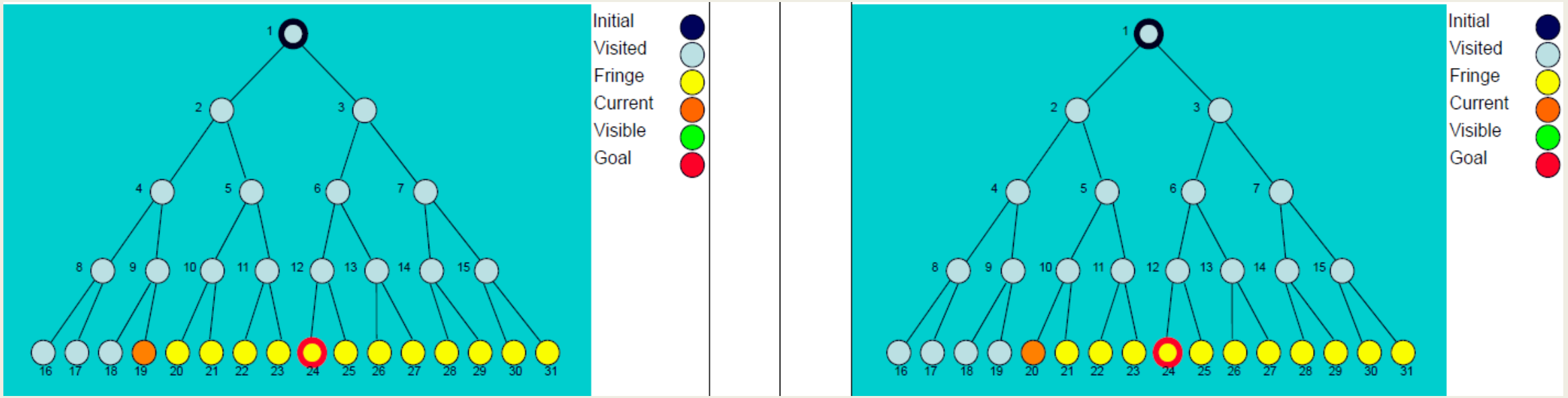




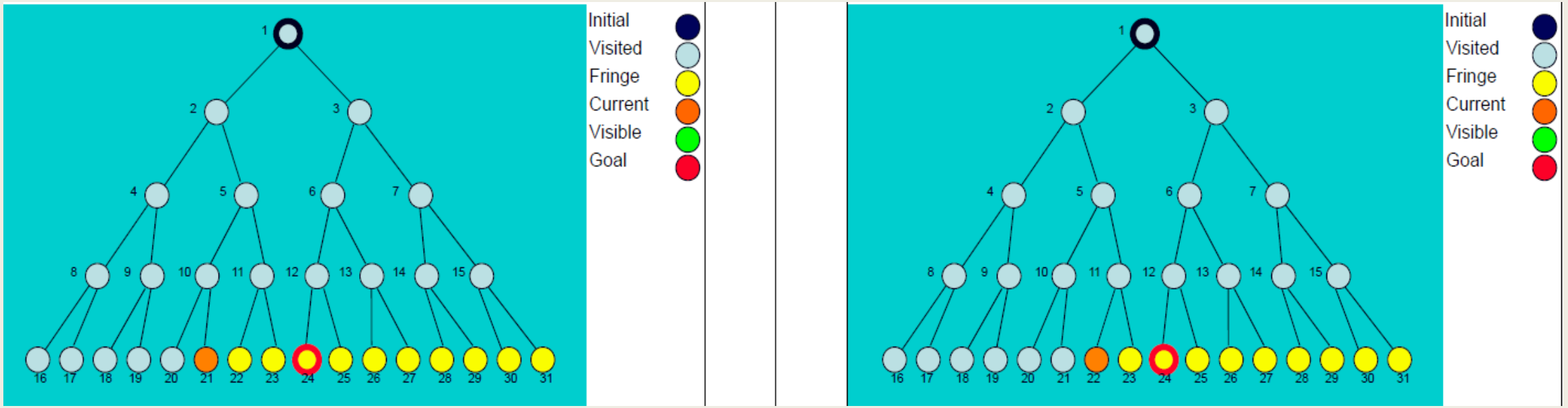
Breadth-First Snapshot



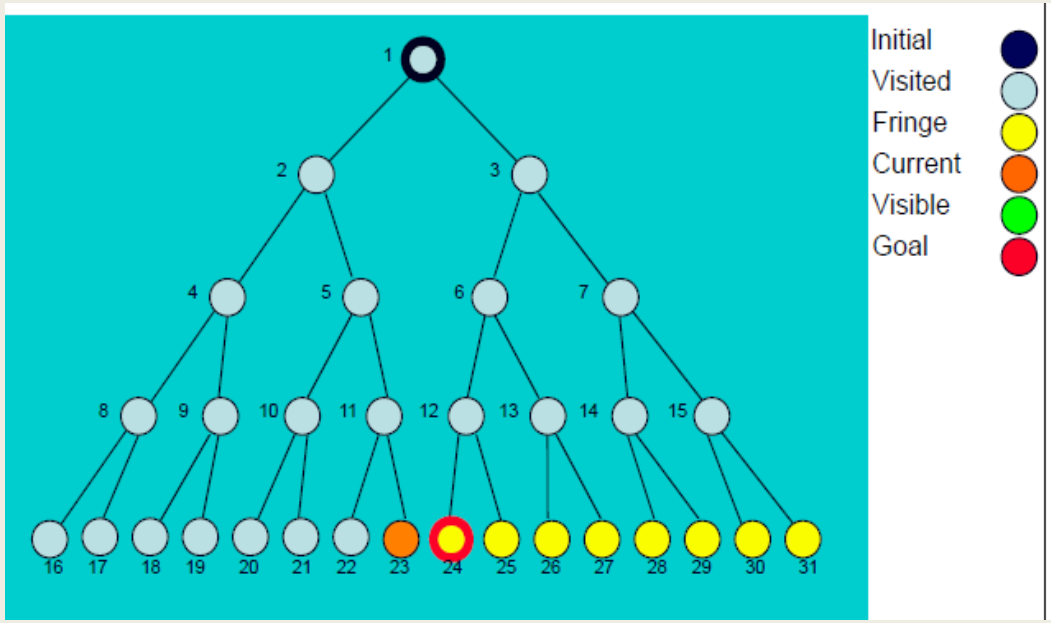
Breadth-First Snapshot

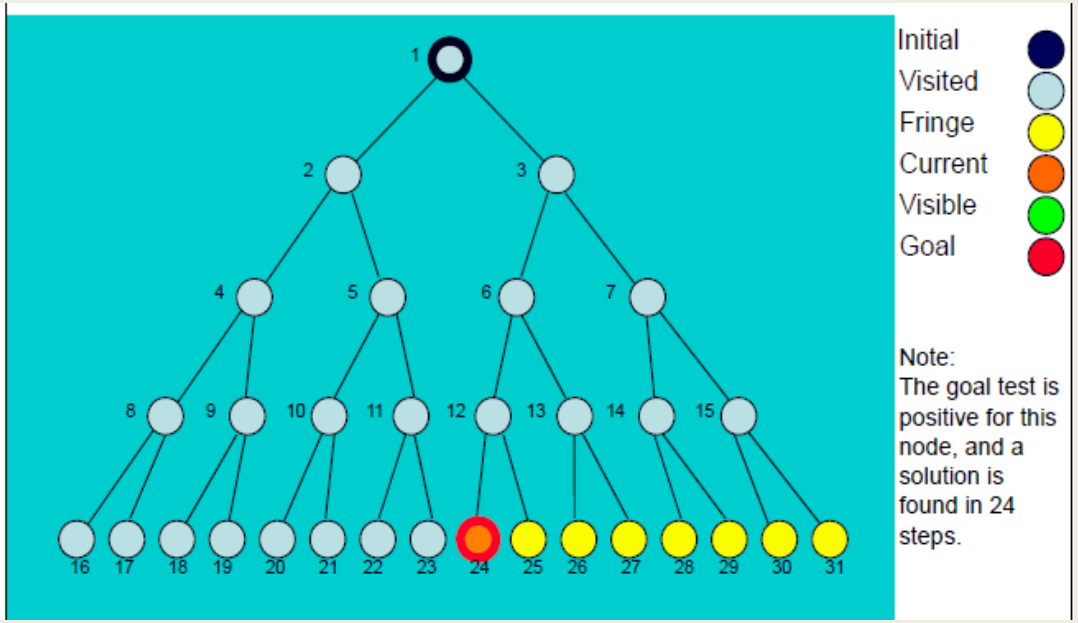


Breadth-First Snapshot

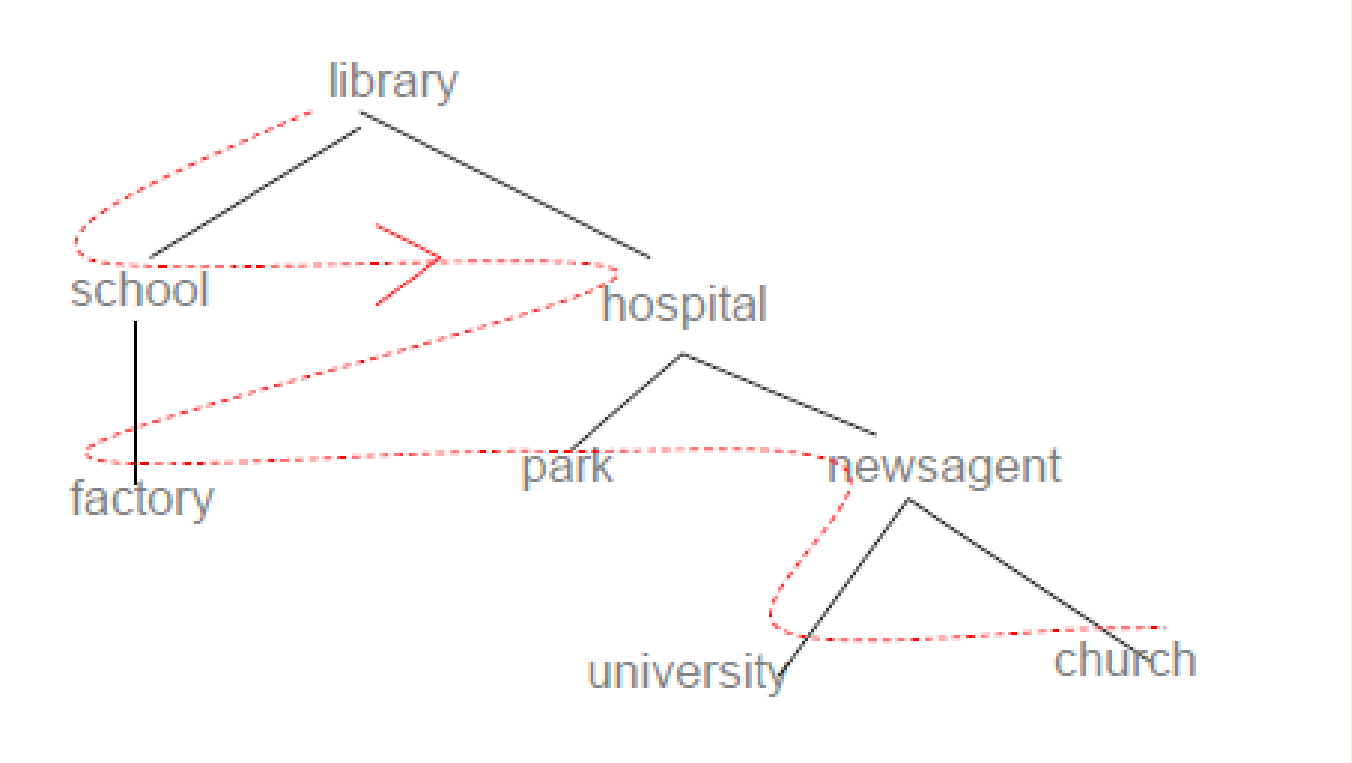


Breadth-First Snapshot





BFS Tree for Route



Pros & Cons of BFS

Pros:

- If there is a solution, BFS is guaranteed to find it.
- If there are multiple solutions, then a minimal solution will be found.
- BFS will not get trapped during exploring.

Cons:

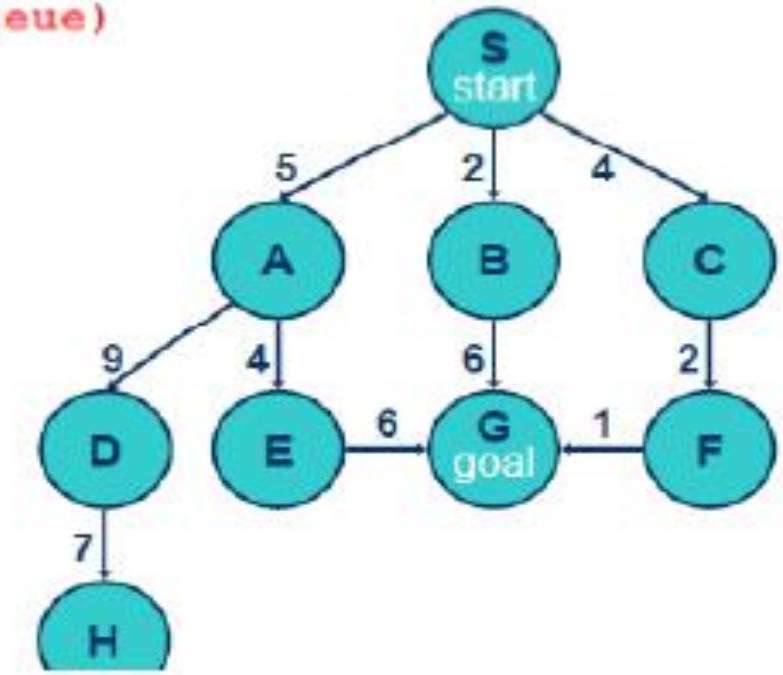
- Each level of nodes are stored before creating the next one, hence it uses more memory.
- If the solution is far away from the root, BFS will consume more time.

Breadth-First Search

generalSearch(problem, queue)

of nodes tested: 0, expanded: 0

expnd. node	nodes list
	{S}

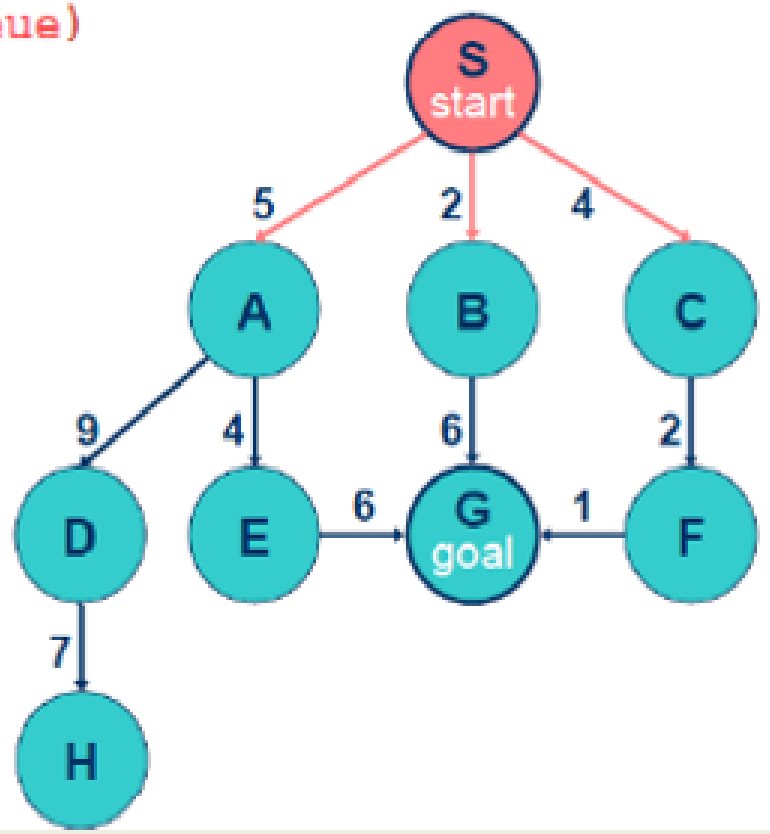


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 1, expanded: 1

expnd. node	nodes list
	{S}
S not goal	{A,B,C}

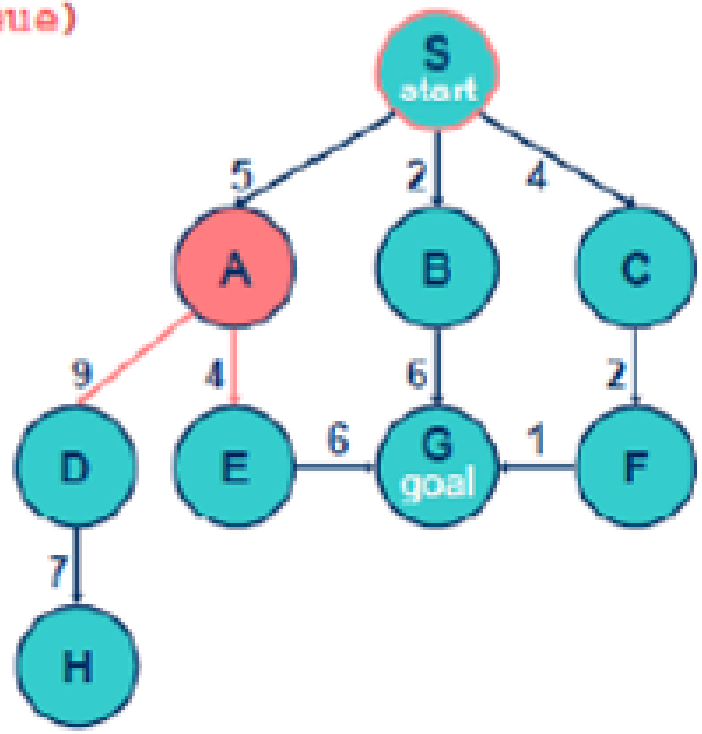


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 2, expanded: 2

expnd. node	nodes list
	{S}
S	{A,B,C}
A not goal	{B,C,D,E}

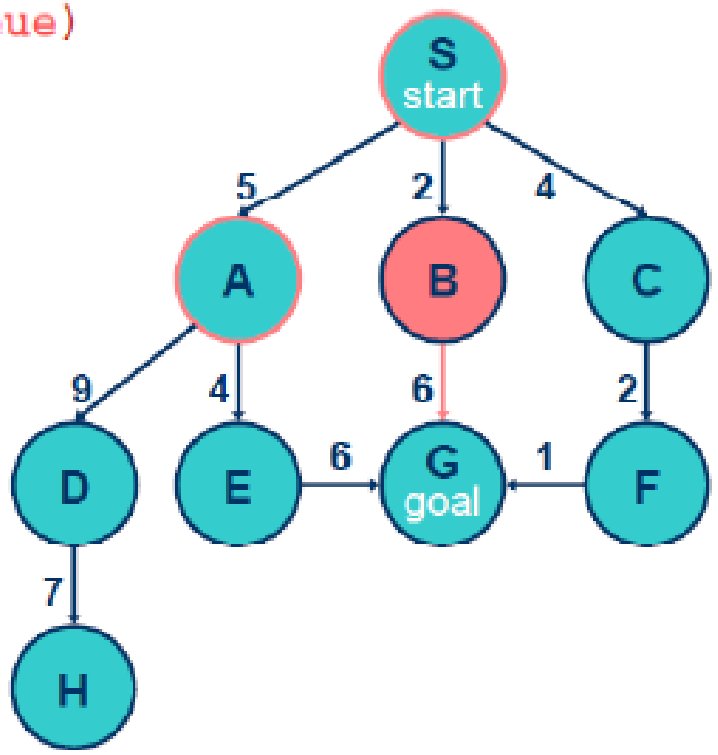


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 3, expanded: 3

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B not goal	{C,D,E,G}

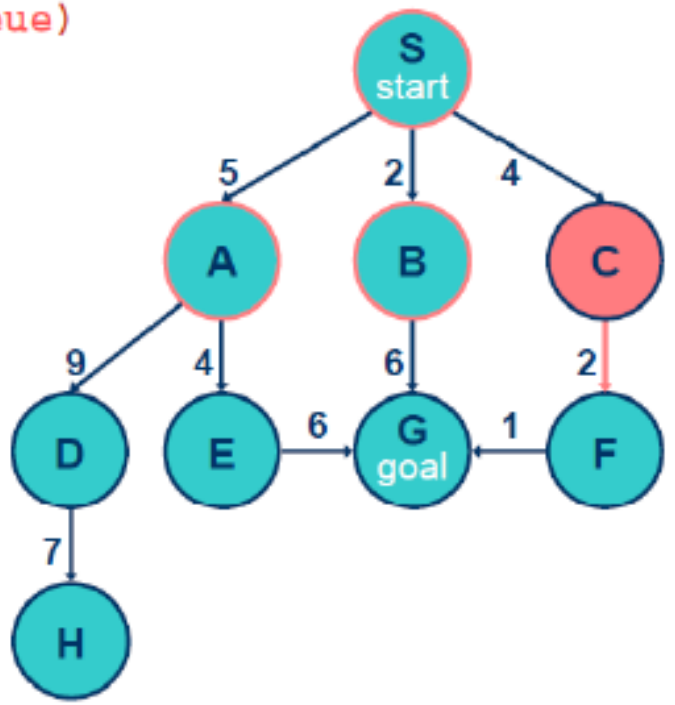


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 4, expanded: 4

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C not goal	{D,E,G,F}

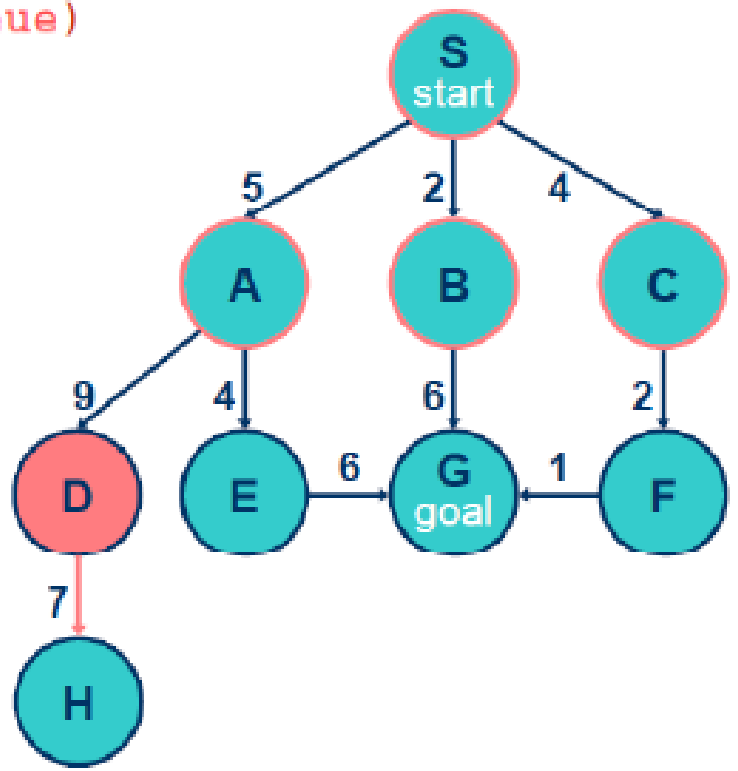


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 5, expanded: 5

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C	{D,E,G,F}
D not goal	{E,G,F,H}

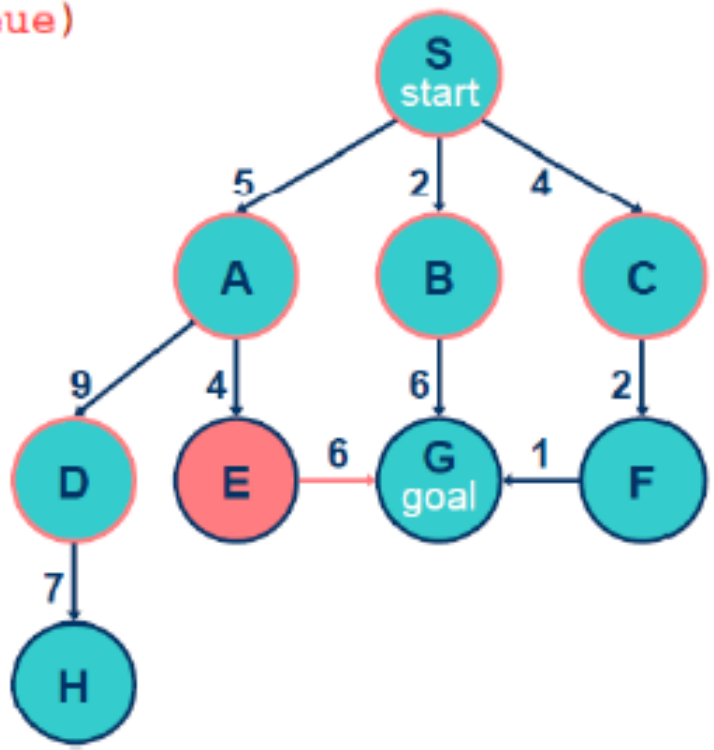


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 6, expanded: 6

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C	{D,E,G,F}
D	{E,G,F,H}
E not goal	{G,F,H,G}

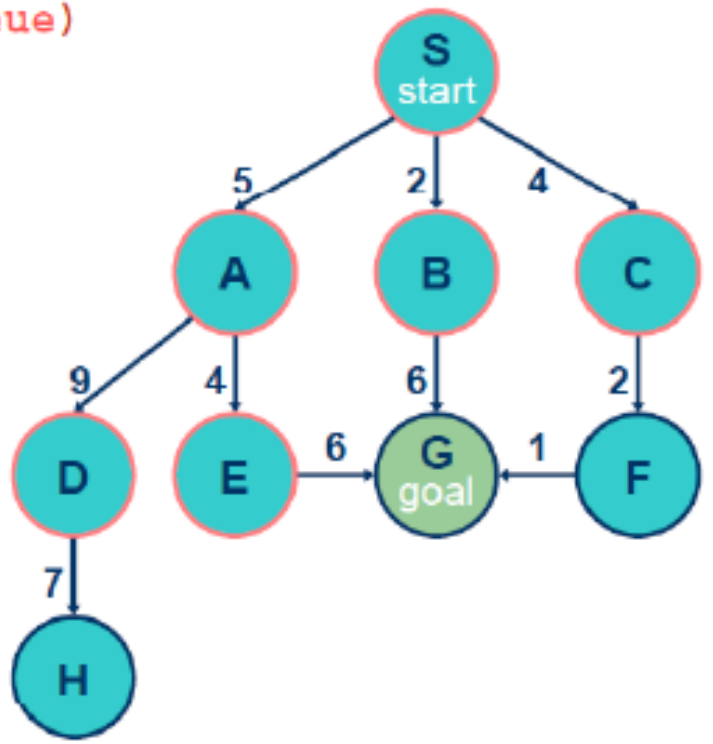


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 7, expanded: 6

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C	{D,E,G,F}
D	{E,G,F,H}
E	{G,F,H,G}
G goal	{F,H,G} no expand

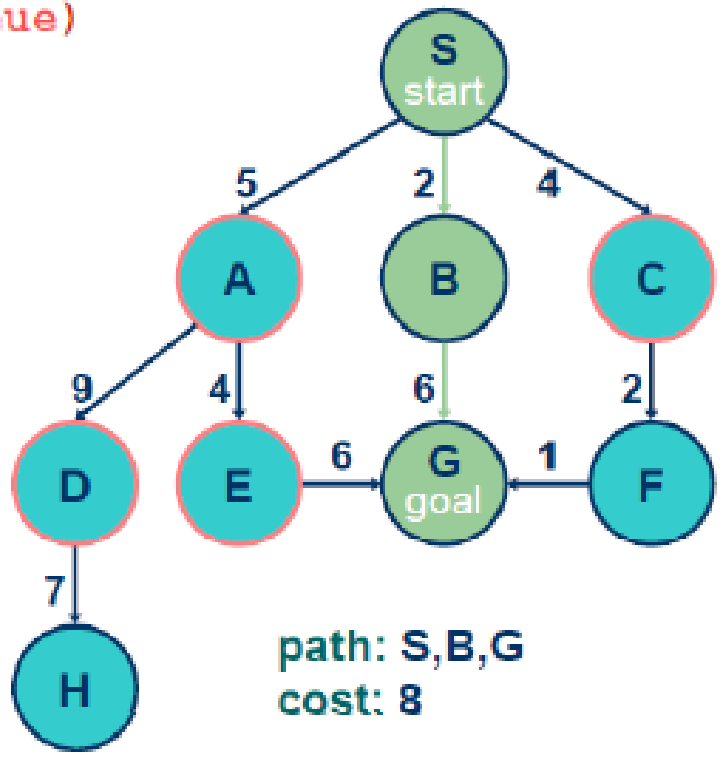


Breadth-First Search

`generalSearch(problem, queue)`

of nodes tested: 7, expanded: 6

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{B,C,D,E}
B	{C,D,E,G}
C	{D,E,G,F}
D	{E,G,F,H}
E	{G,F,H,G}
G	{F,H,G}



Properties of BFS

- **Completeness:**

Yes, the solution is reached if it exists.

- **Optimality:**

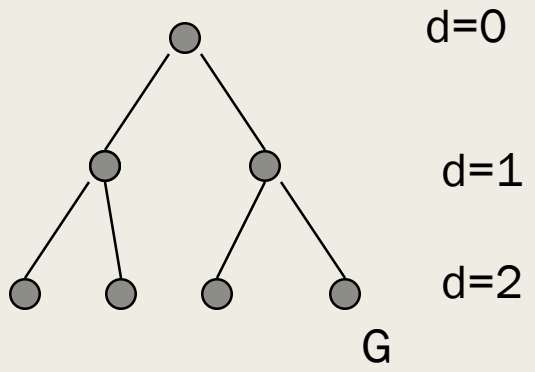
Yes.

Properties of BFS

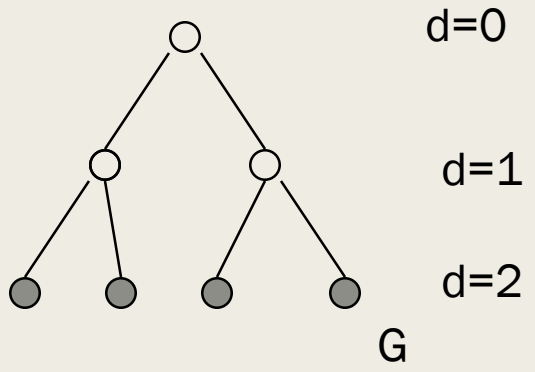
- **Time Complexity**
  - assume (worst case) that there is 1 goal leaf at G
  - so BFS will expand all nodes

$= 1 + b + b^2 + \ldots\ldots\ldots + b^d$

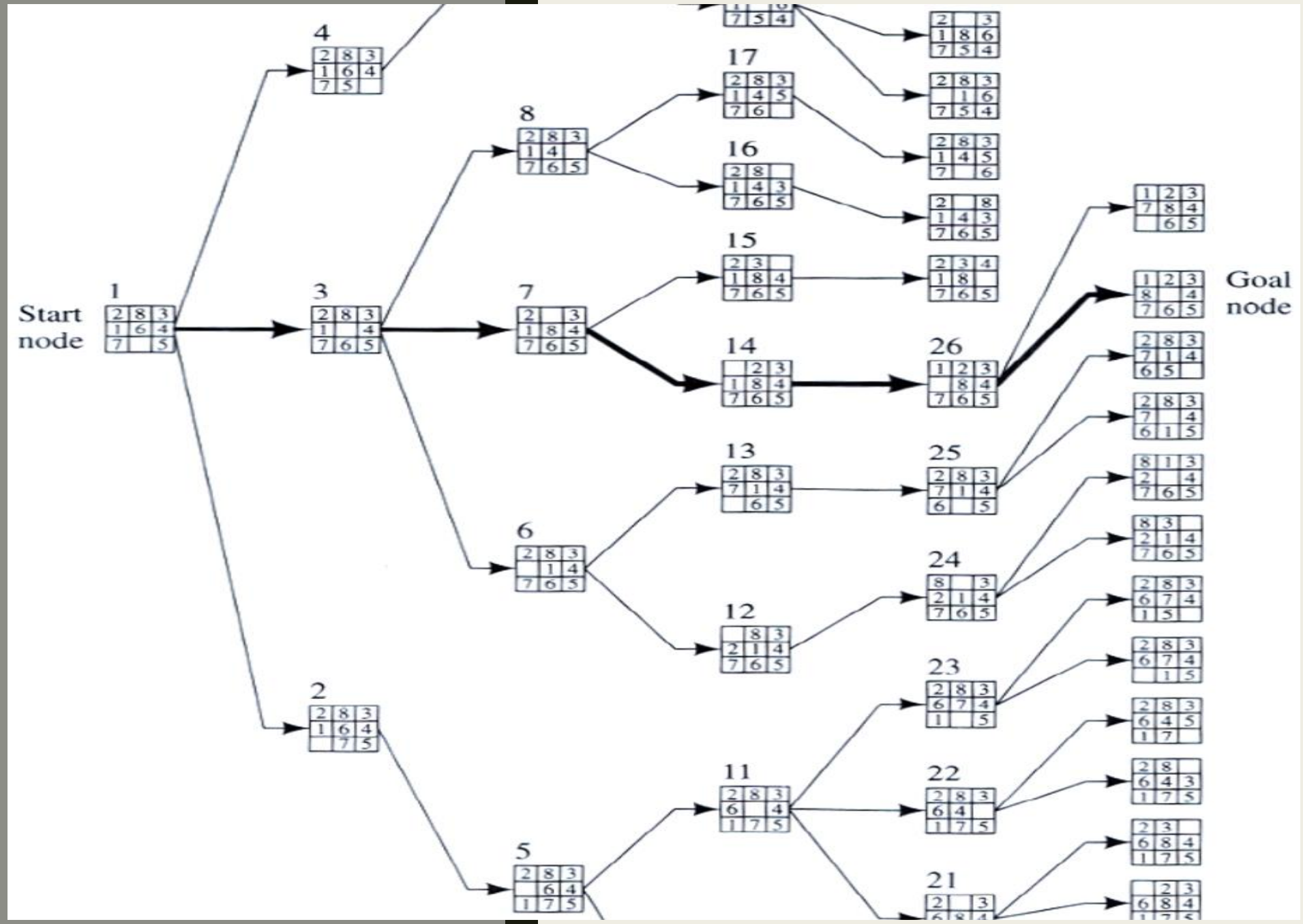
$= \mathbf{O(b^d)}$



- **Space Complexity**
  - how many nodes can be in the queue (worst-case)?
  - at depth d-1 there are b^d unexpanded nodes in the Q = $\mathbf{O(b^d)}$
  - Nodes are kept in memory.



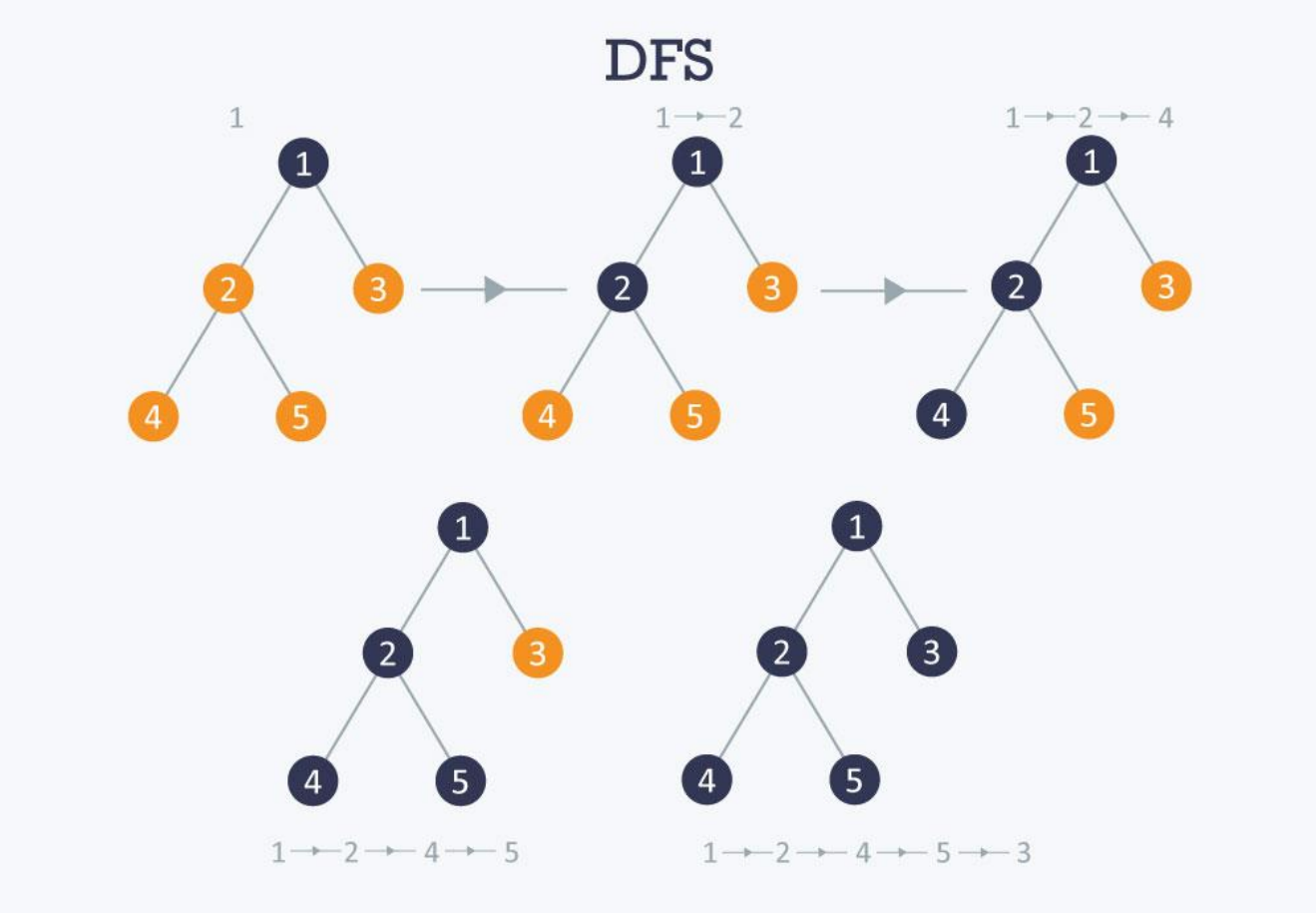
BFS Example


Start node
Goal node
1
2
3
4
5
6
7
8
11
12
13
14
15
16
17
21
22
23
24
25
26

Depth-first Search

- Expand deepest unexpanded node
- DFS progresses by expanding the first child node of the search tree that appears and thus going deeper and deeper until a goal node is found, or until it hits a node that has no children. Then the search backtracks, returning to the most recent node it hasn't finished exploring.
- Implementation:
  - A **LIFO** data structure, i.e., a stack
- Guarantees to find a solution if the problem space is finite.
  - achieved by the TREE-SEARCH method by appending newly generated nodes at the beginning of stack.

Depth-first Search

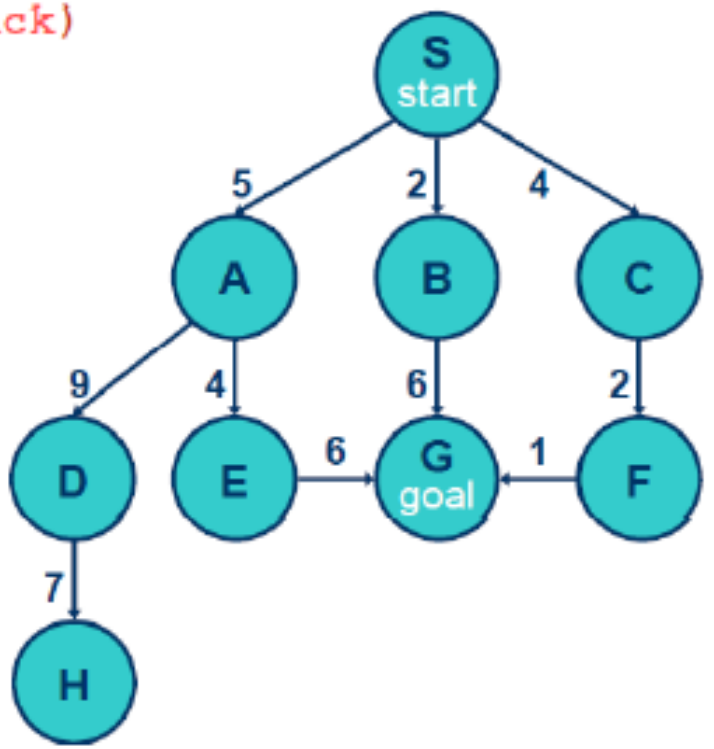


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 0, expanded: 0

expnd. node	nodes list
	{S}

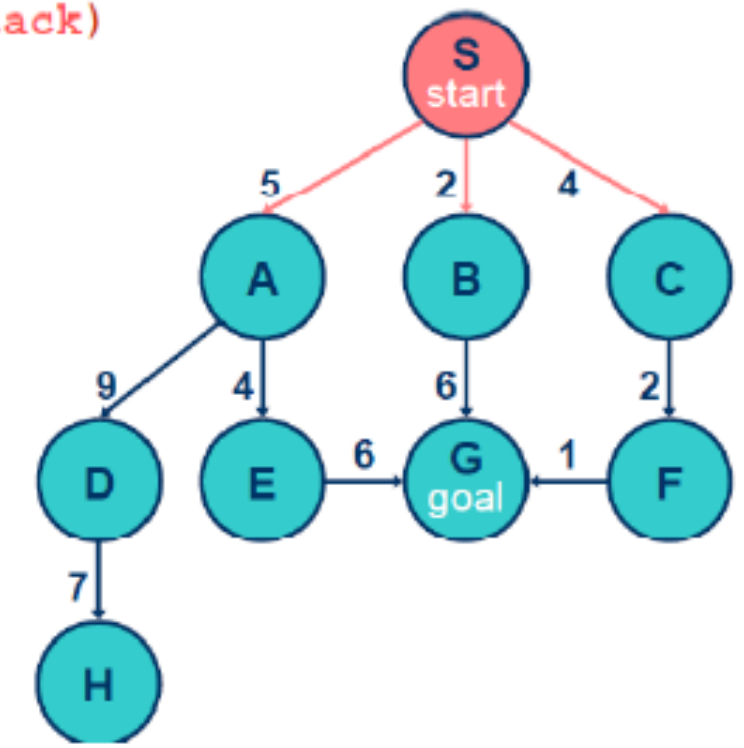


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 1, expanded: 1

expnd. node	nodes list
	{S}
S not goal	{A,B,C}

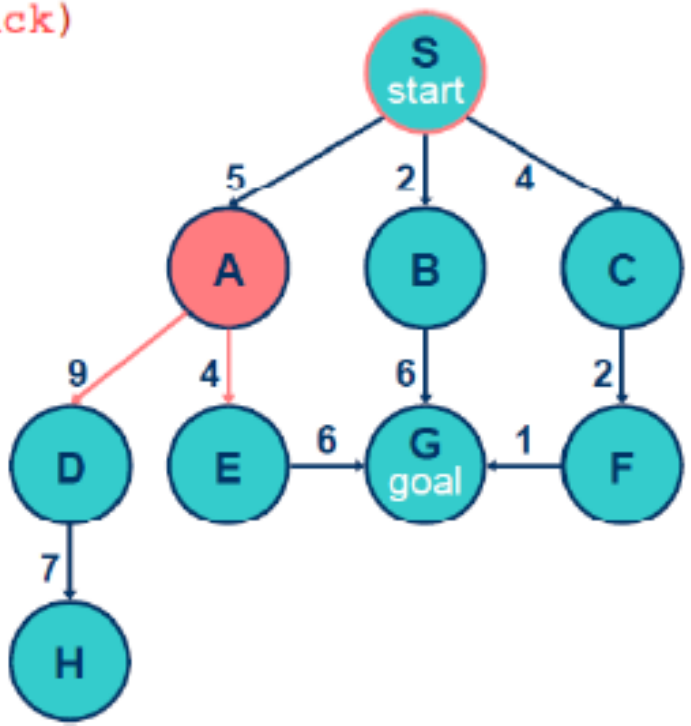


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 2, expanded: 2

expnd. node	nodes list
	{S}
S	{A,B,C}
A not goal	{D,E,B,C}

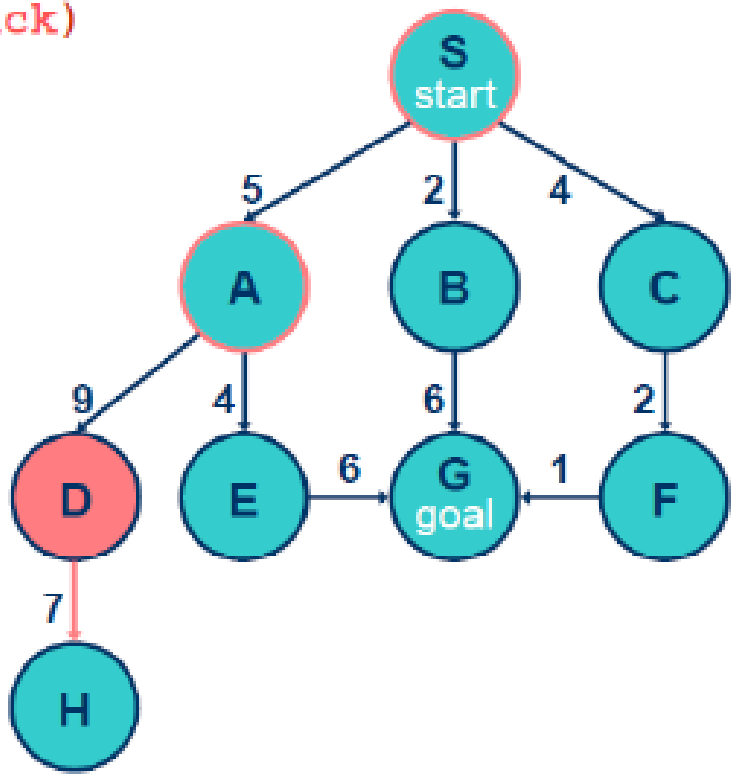


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 3, expanded: 3

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{D,E,B,C}
D not goal	{H,E,B,C}

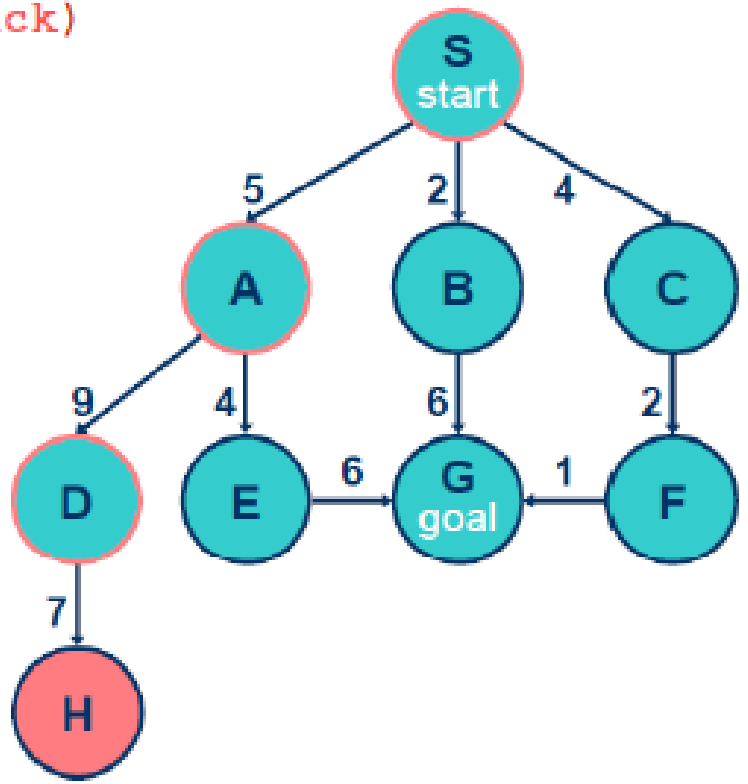


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 4, expanded: 4

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H not goal	{E,B,C}

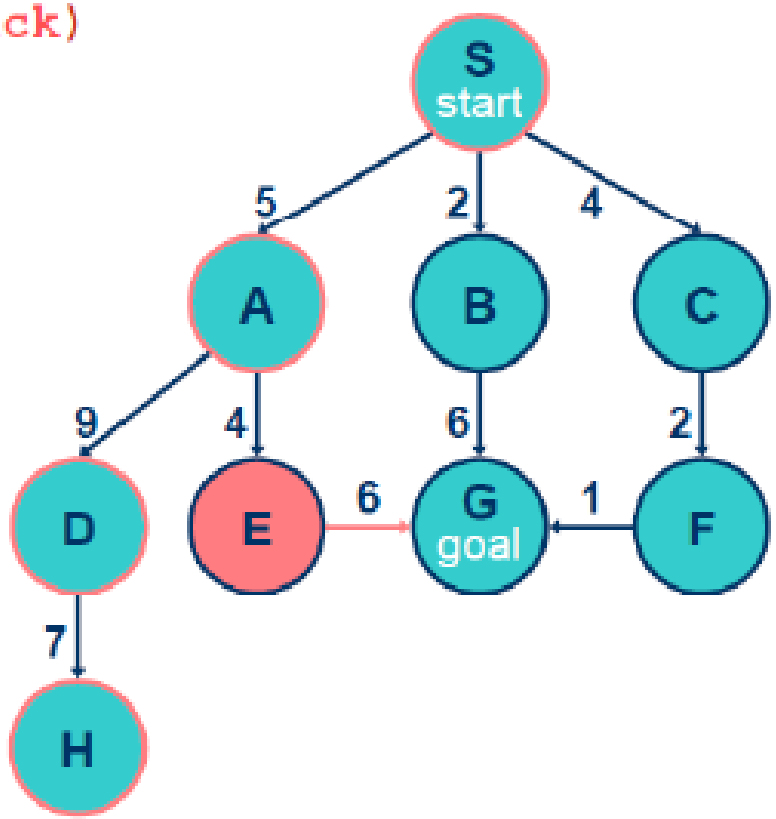


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 5, expanded: 5

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H	{E,B,C}
E not goal	{G,B,C}

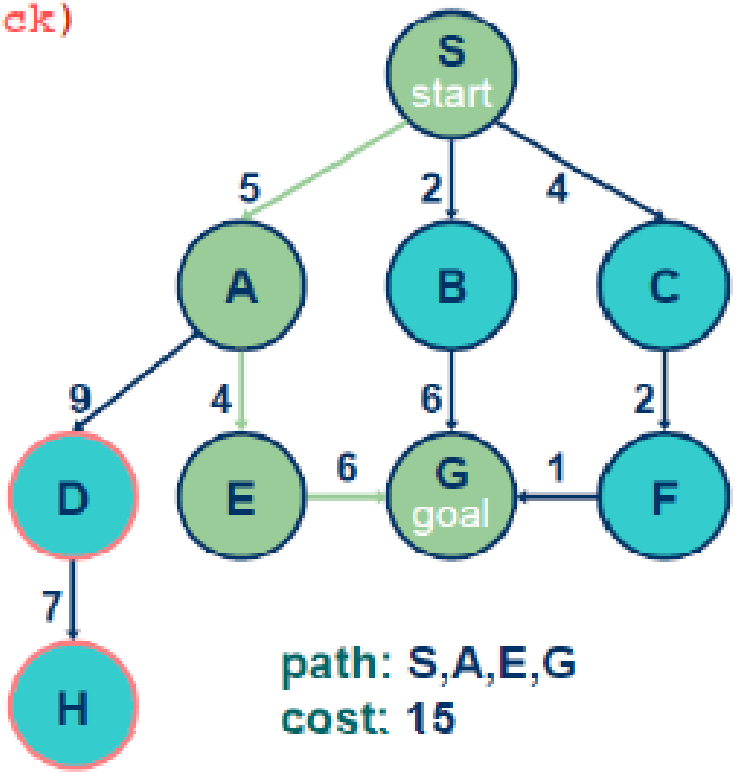


Depth-first Search

`generalSearch(problem, stack)`

of nodes tested: 6, expanded: 5

expnd. node	nodes list
	{S}
S	{A,B,C}
A	{D,E,B,C}
D	{H,E,B,C}
H	{E,B,C}
E	{G,B,C}
G	{B,C}



Pros and Cons of DFS

Pros

- DFS requires less memory since only the nodes on the current path are stored.
- If DFS finds solution without exploring much in a path then the time and space it takes will be very less.

Cons

- The disadvantage of Depth-First Search is that there is a possibility that it may go down to infinite loop when there is a cycle.
- Depth-First Search is incomplete when the search space is not finite, thus not guaranteed to find the solution.

Properties of DFS

- **Complete?**

No (Yes on finite trees, fails in infinite state space)

- **Optimality:**

Consider the scenario that there is more than one goal node, and our search decided to first expand the left sub tree of the root where there is a solution at a very deep level of this left sub tree, in the same time the right sub tree of the root has a solution near the root, here comes the non-optimality of DFS that it is not guaranteed that the first goal to find is the optimal one, so we conclude that DFS is non-optimal.

Properties of DFS

- **Time Complexity**
  - *assume (worst case) that there is 1 goal leaf at G so, DFS will expand all nodes*

$$=1 + b + b^2 + \ldots\ldots\ldots + b^d$$
$$= O(b^d)$$

  - *bad if d is much larger than b*
- **Space Complexity**
  - *how many nodes in the stack?*

level 1: b-1 nodes

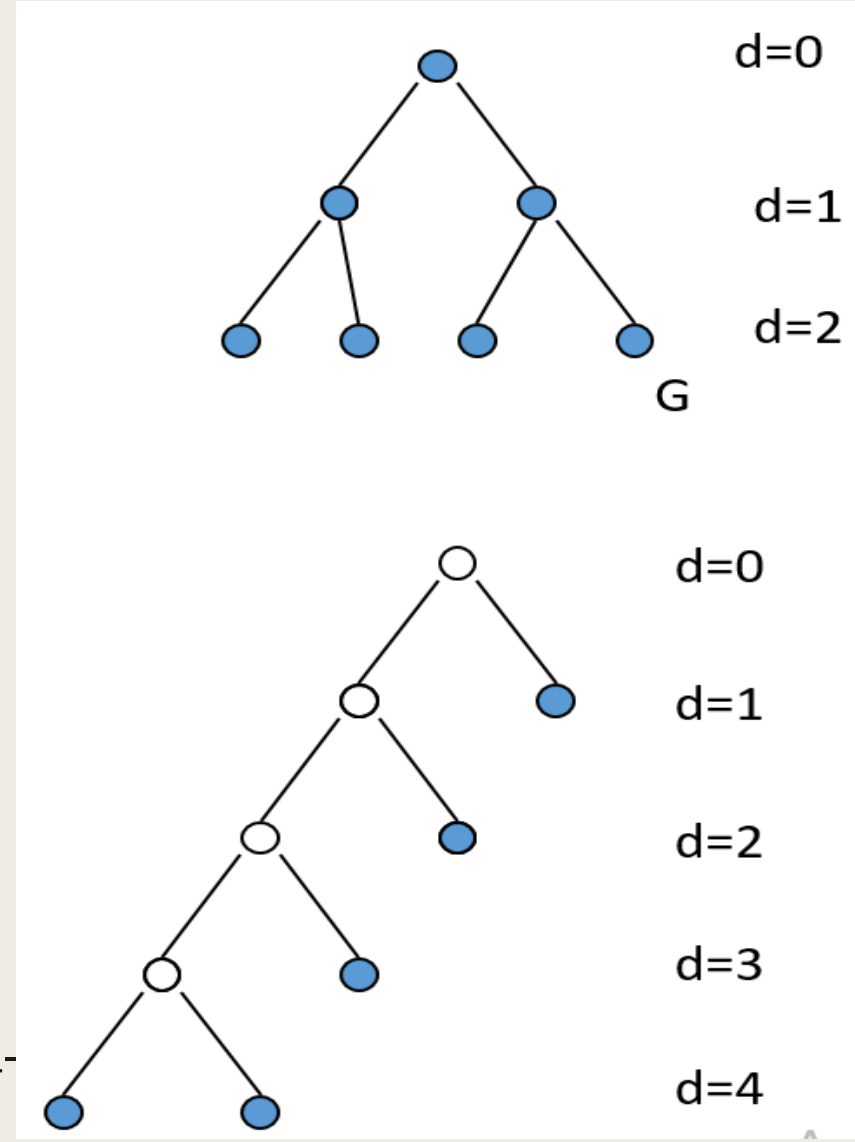
level 2 : b-1 nodes

....

level d-1: b-1 nodes

level d: b nodes

- So, total node in the memory = (b-1)(d-1)+b = bd+1-b+d+b = bd+1-d = (b-1)d+1 === **aprroximately O(bd) space.**



BFS vs DFS

Breadth First Search (BFS)

- BFS visit nodes **level by level** in Graph.
- A node is fully explored before any other can begin.
- Uses Queue data structure to store Un-explored nodes.
- BFS require more memory.
- No backtracking is required.
- If search target is concentrated more near the root and not very deep, BFS should perform better and vice-versa.

Depth First Search (DFS)

- DFS visit nodes of graph **depth wise**. It visits nodes until reach a leaf or a node which doesn't have non-visited nodes.
- Exploration of a node is suspended as soon as another unexplored is found.
- Uses Stack data structure to store Un-explored nodes.
- DFS require less memory.
- Backtracking is required.
- If search target distribution is frequent in the search-space, DFS should perform better and vice-versa.

Depth-Limited Search

- It is depth-first search with an imposed limit on the depth of exploration, to guarantee that the algorithm ends.
- Simply put an upper limit on the depth of paths allowed.

  - Expand node at the deepest level, but limit depth to L
- Motivation:

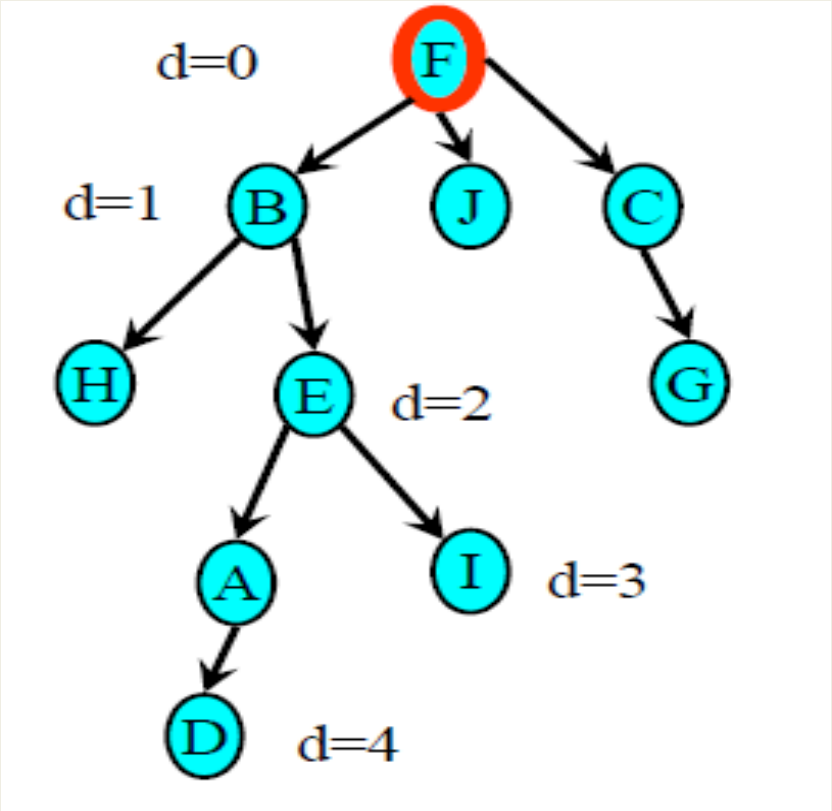
  - Prevents search diving into deep solutions

  - Might already have a solution of known depth (cost), but are looking

    for a shallower (cheaper) one

Depth-Limited Search

- Depth limit of 2 would mean that children of E are ignored.



Properties of Depth-Limited Search

- **Complete?**

Yes if there is a goal state at a depth less than L

- **Optimal?**

No

- **Time Complexity:**

O (b^L), *where L is the cutoff.*

- **Space Complexity:**

O(bL), where L is the cutoff.

Iterative Deepening Search or IDDFS

- **BFS**

– good for optimality

– *bad on memory, $O(b^d)$*

- **DFS**

– solutions not guaranteed optimal: dives and misses good nodes

– good for memory O(bd)

• "Iterative Deepening" refers to a method that tries to combine the best of the above.

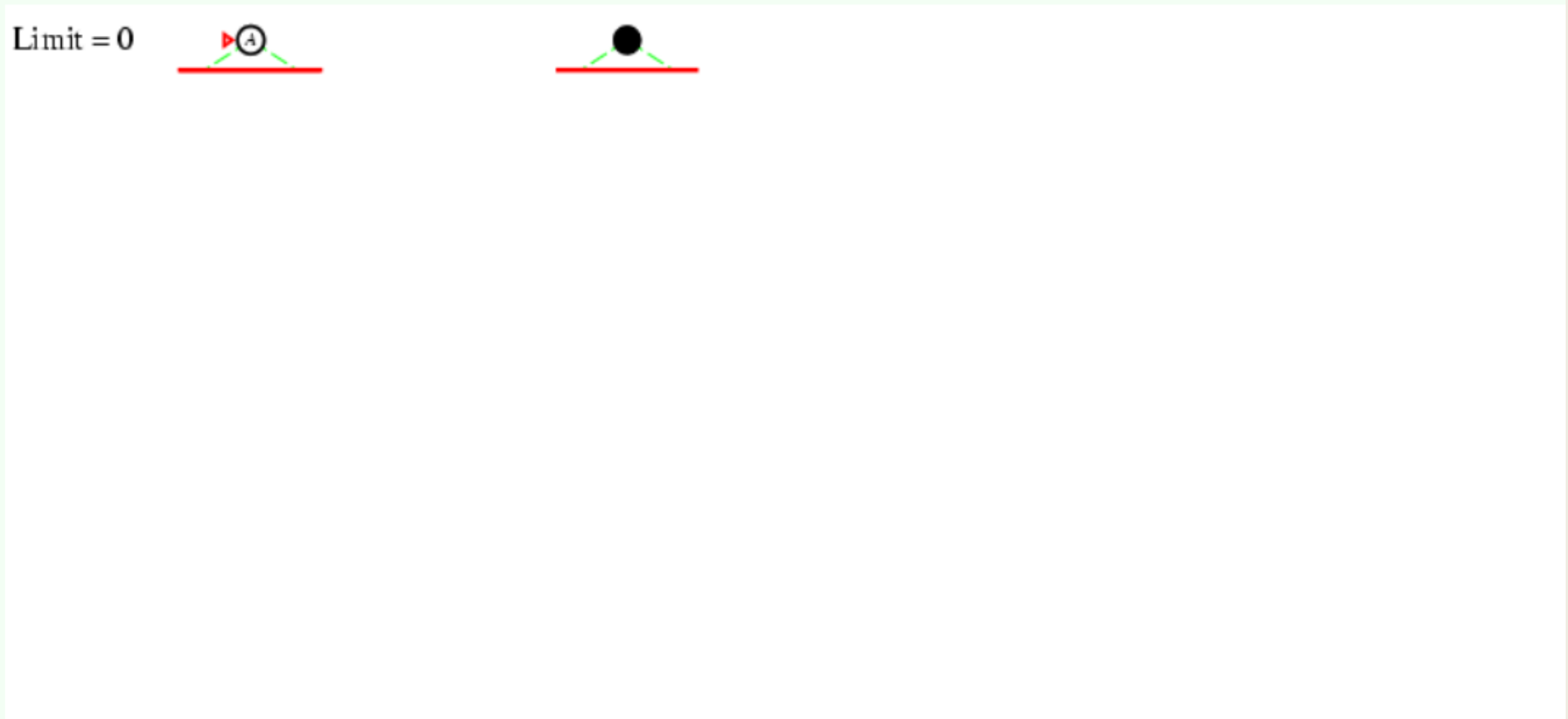
Iterative Deepening Search

- To avoid the infinite depth problem of DFS, we can decide to only search until depth L, i.e. we don't expand beyond depth L. That is Depth-Limited Search.
- What if solution is deeper than L? Ans: Increase L iteratively.

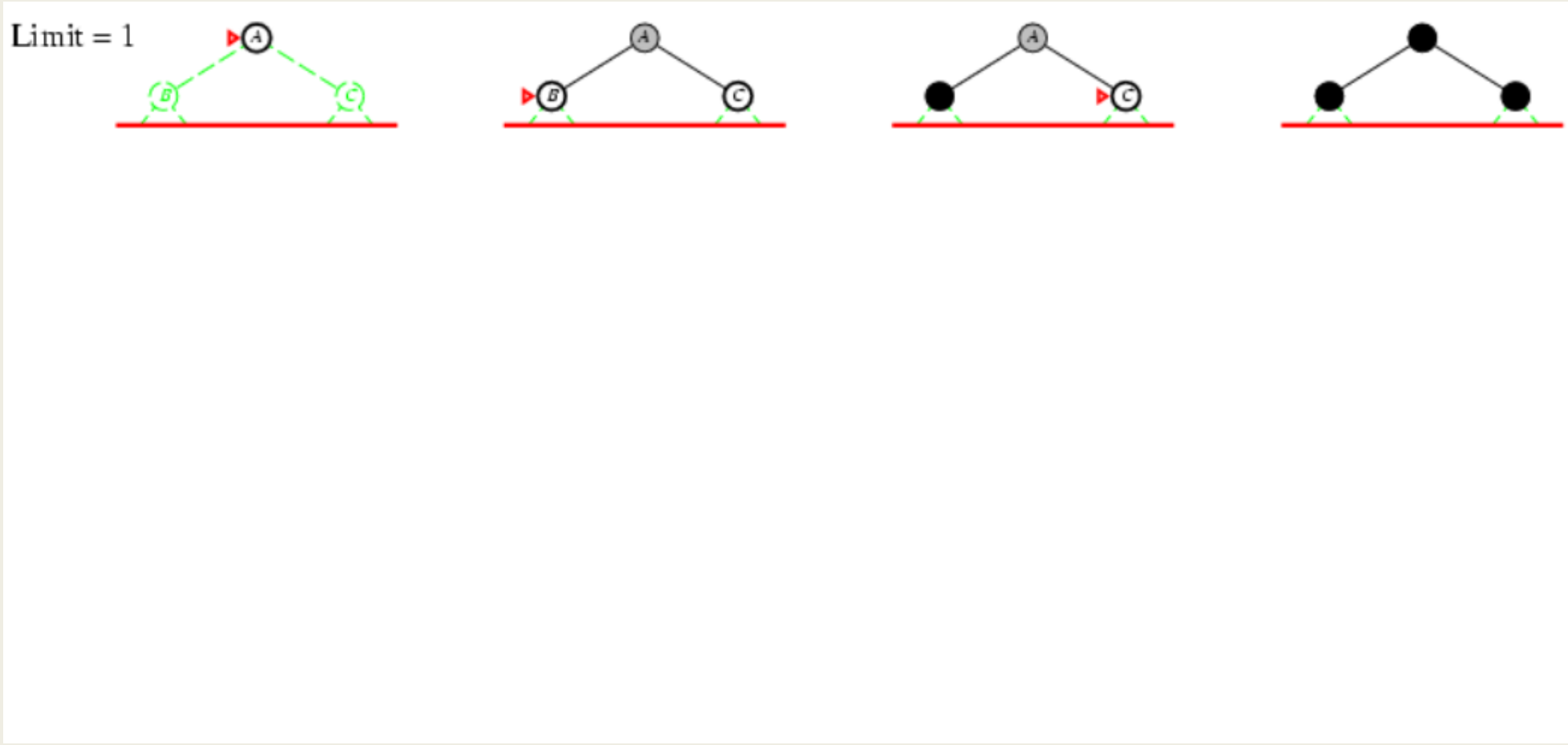
❖ **Iterative Deepening Search**

- As we'll see: this inherits the memory advantage of Depth-First search, and is better in terms of time complexity than Breadth first search.
- We get the benefits of BFS and DFS through this process.

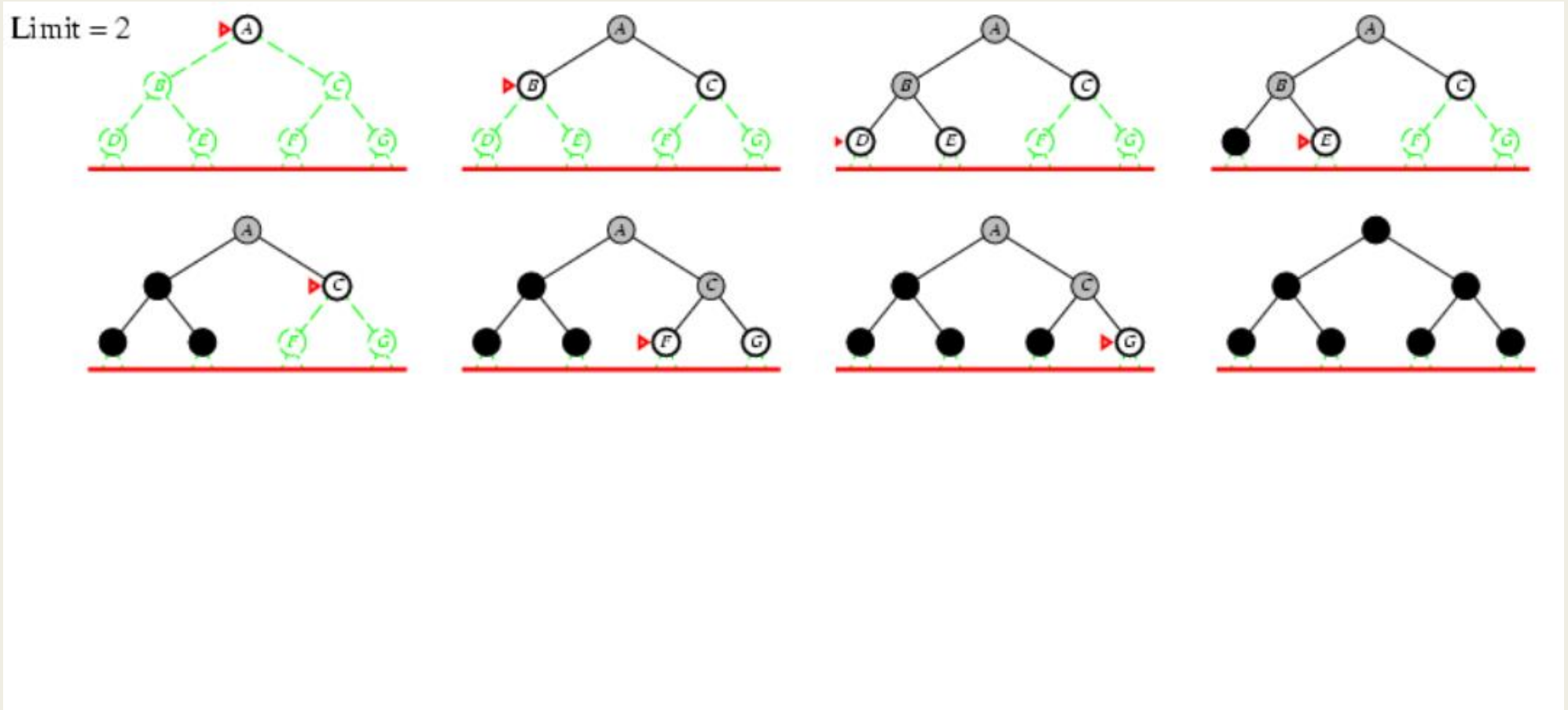
Iterative Deepening Search L=0



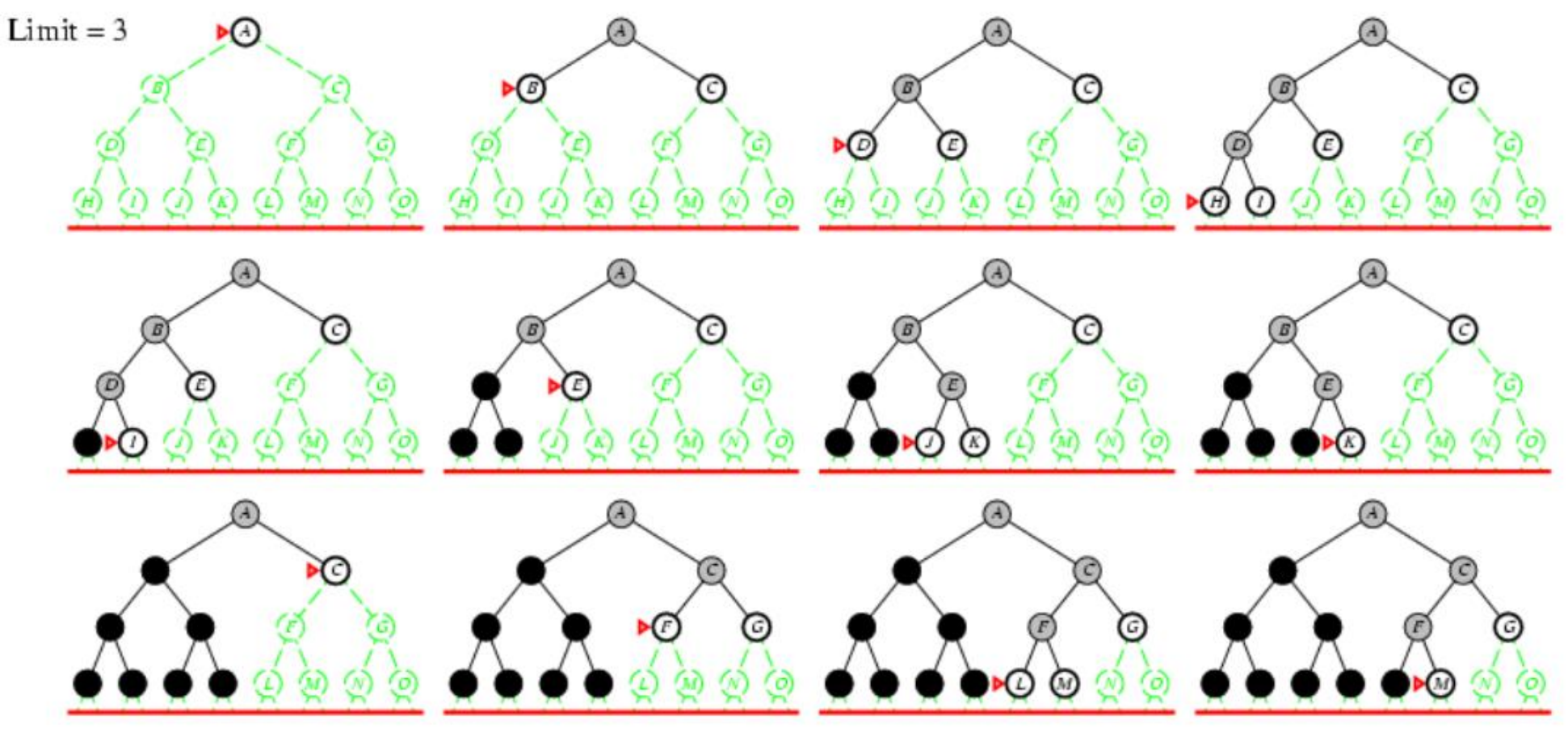
Iterative Deepening Search L=1



Iterative Deepening Search L=2



Iterative Deepening Search L=3



Iterative Deepening Search

- **Complete?**

Yes

- **Optimal?**

Yes

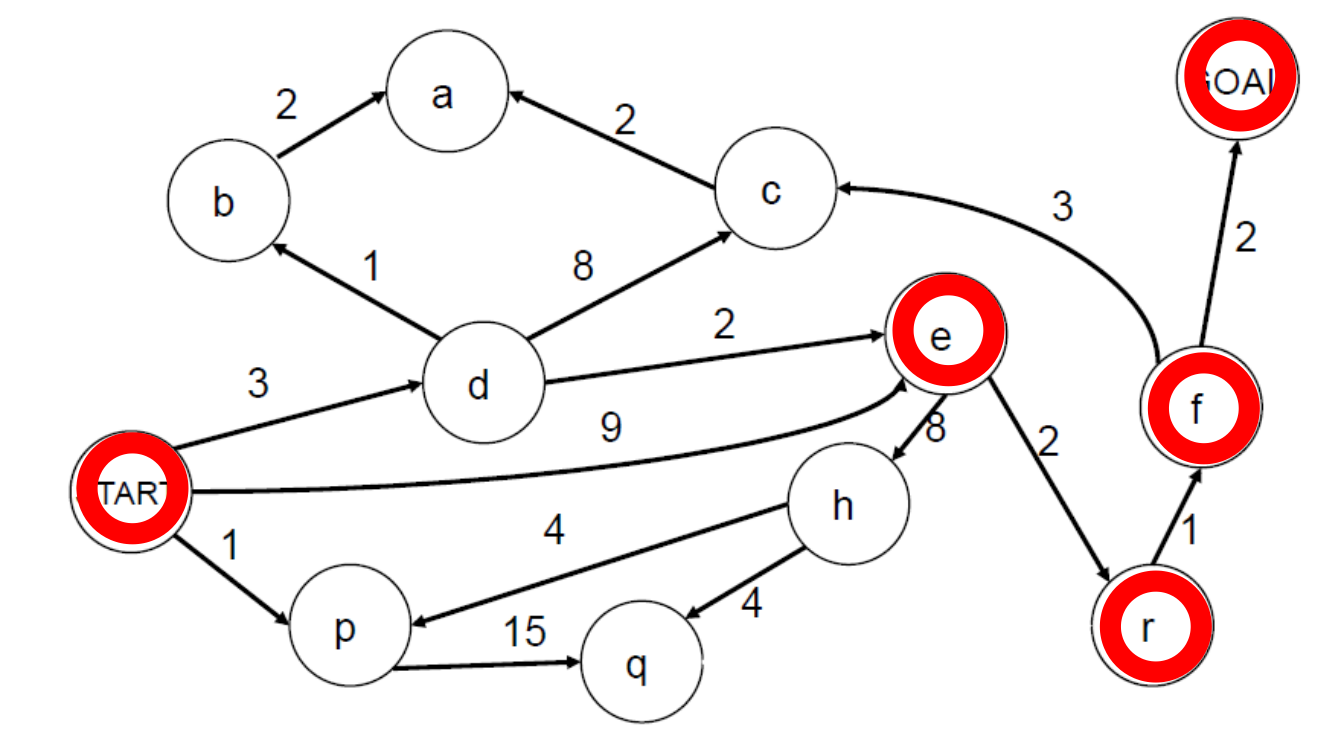
- **Time**

$O(b^d)$, where d is the depth of the solution.

- **Space**

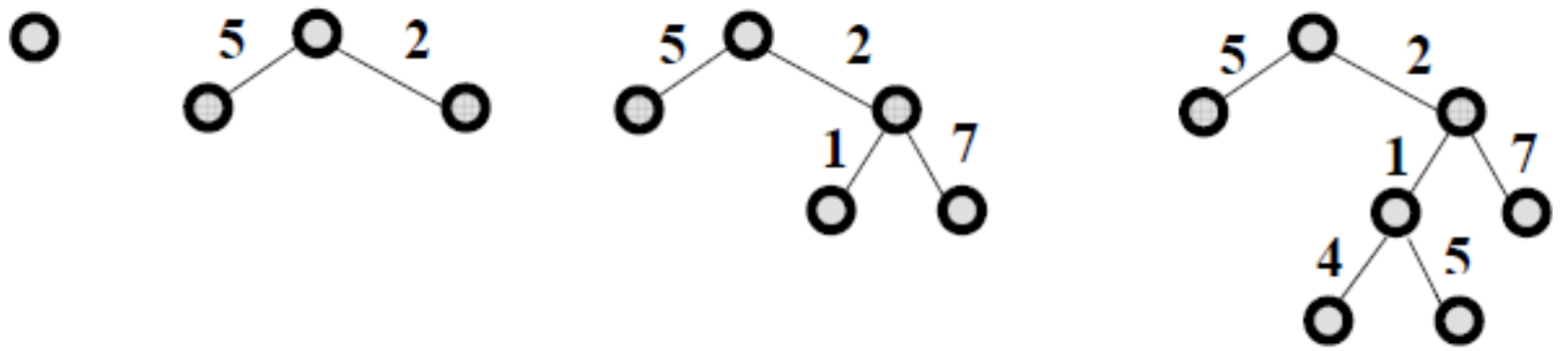
O(bd), where d is the depth of the solution.

Search with varying step costs



- BFS finds the path with the fewest steps, but does not always find the cheapest path.

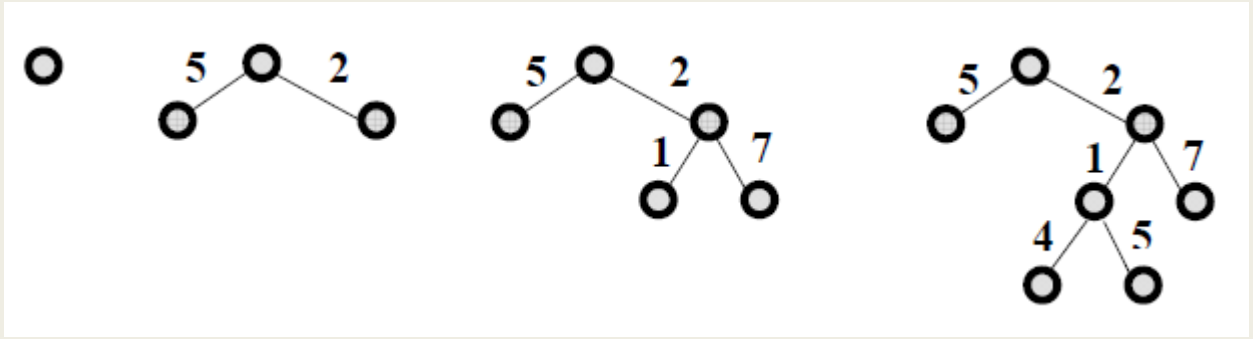
Uniform-Cost Search(UCS)



- Intuition: *Expand the cheapest node. Where the cost is the path cost g(n)*
- Enqueue nodes in order of cost
- Equivalent to breadth-first if step costs all equal

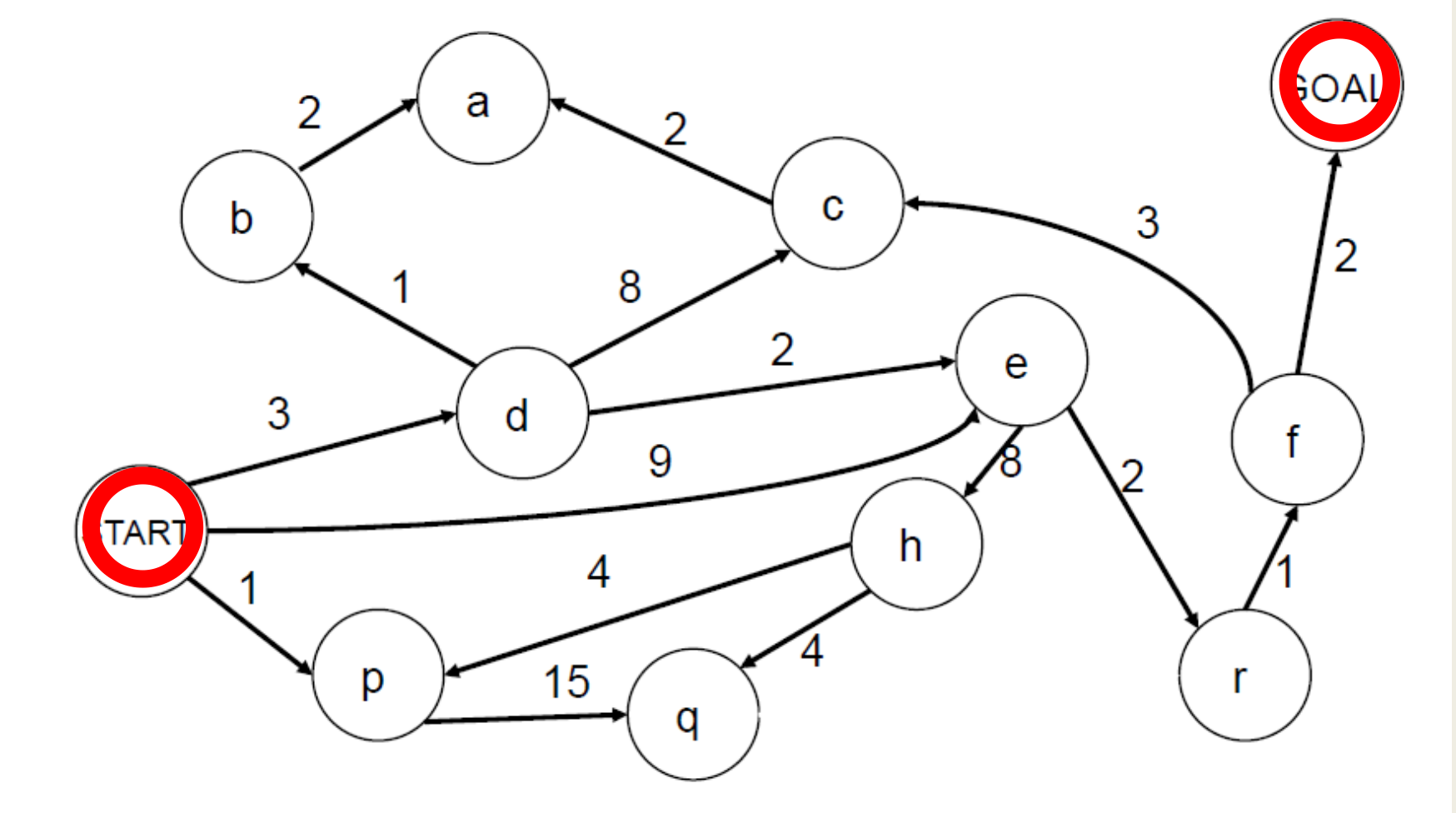
Uniform-Cost Search

- At any given point in the execution, the algorithm never expands a node which has a cost greater than the cost of the shortest path in the tree.
- First goal found is least-cost solution

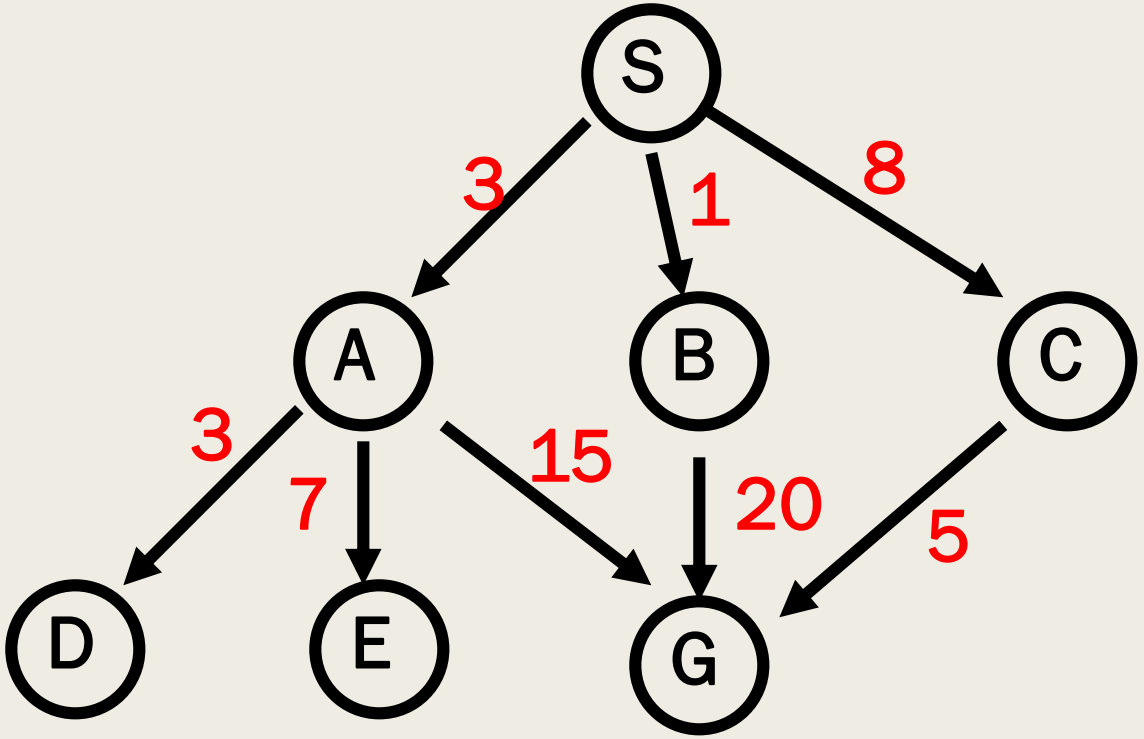


- The nodes are stored in a priority queue.

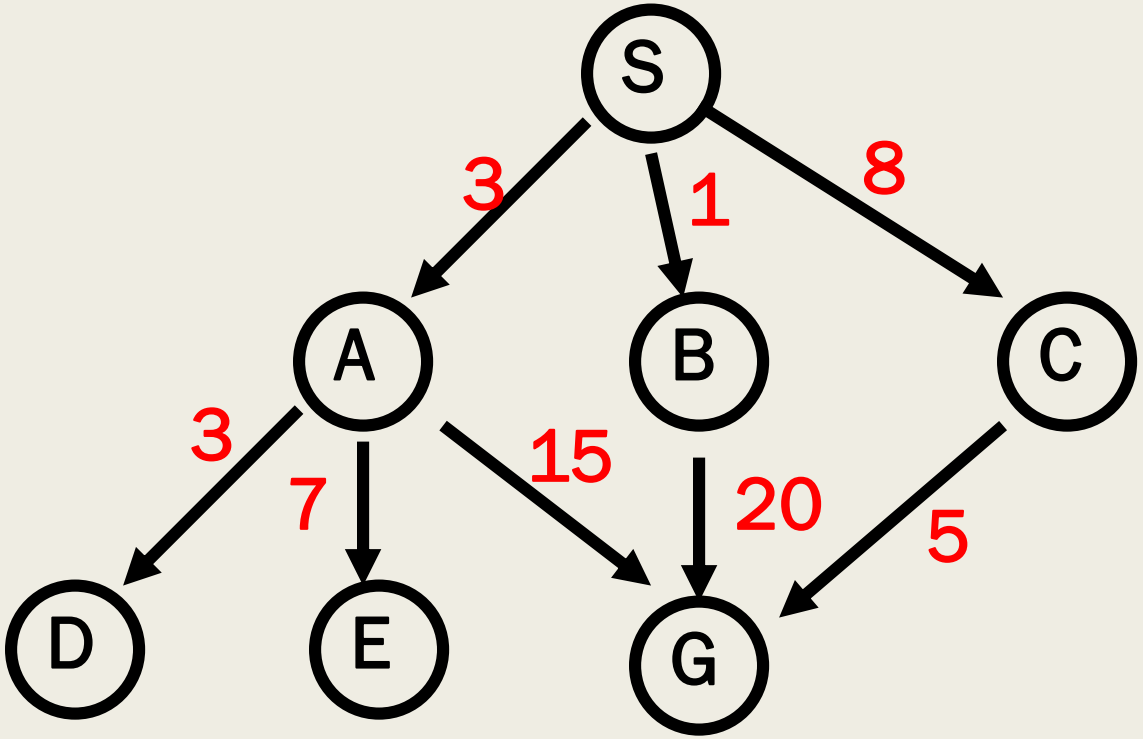
UCS Example



Example for illustrating uninformed search strategies



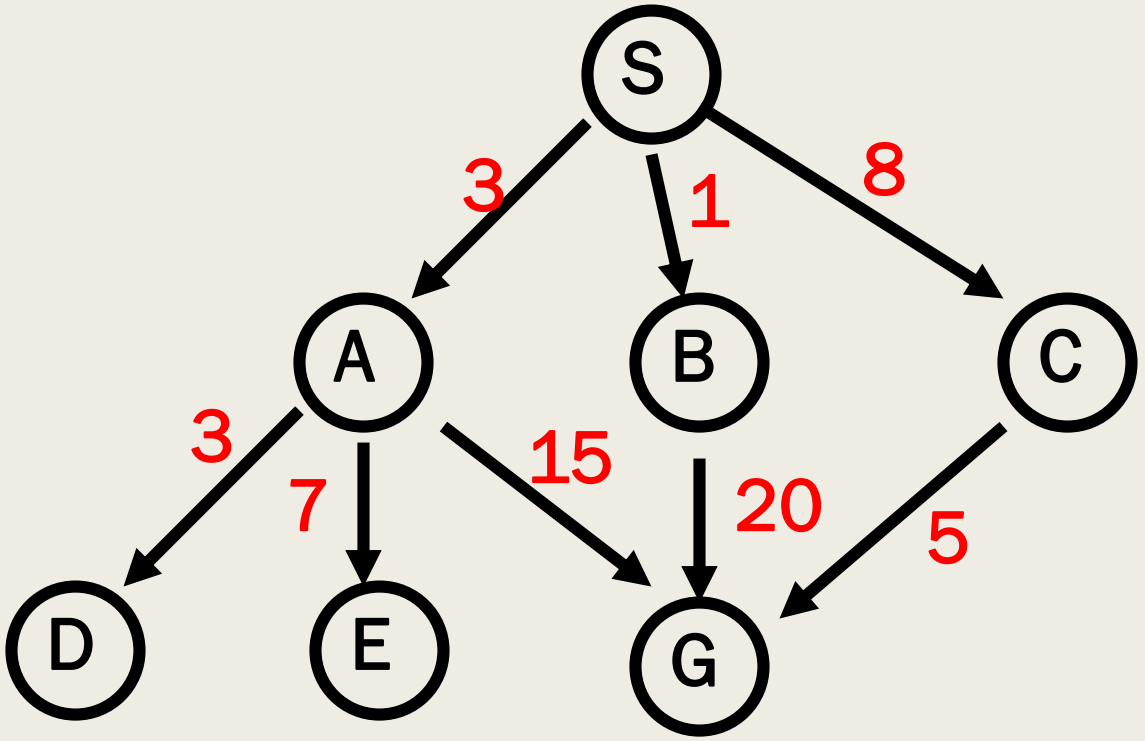
Breadth-First Search



Expanded node	Nodes list
	{ S^0 }
S^0	{ A^3 B^1 C^8 }
A^3	{ B^1 C^8 D^6 E^{10} G^{18} }
B^1	{ C^8 D^6 E^{10} G^{18} G^{21} }
C^8	{ D^6 E^{10} G^{18} G^{21} G^{13} }
D^6	{ E^{10} G^{18} G^{21} G^{13} }
E^{10}	{ G^{18} G^{21} G^{13} }
G^{18}	{ G^{21} G^{13} }

Solution path found is S A G , cost 18

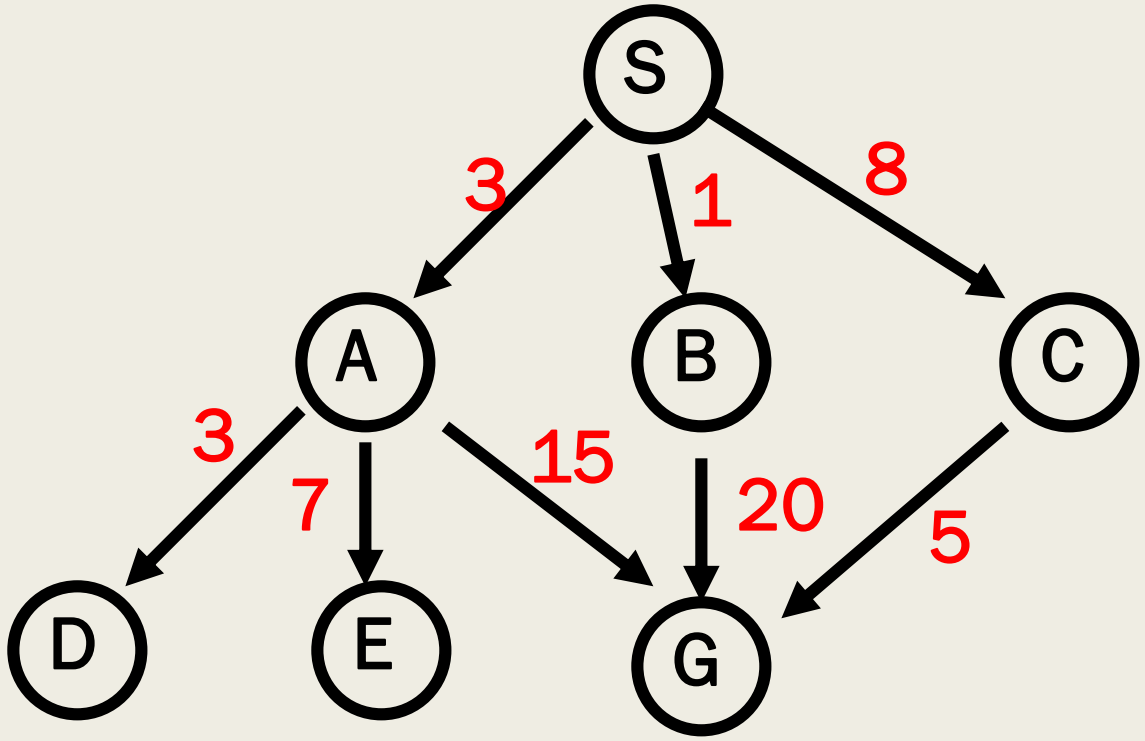
Depth-First Search



Expanded node	Nodes list
	{ S^0 }
S^0	{ A^3 B^1 C^8 }
A^3	{ D^6 E^{10} G^{18} B^1 C^8 }
D^6	{ E^{10} G^{18} B^1 C^8 }
E^{10}	{ G^{18} B^1 C^8 }
G^{18}	{ B^1 C^8 }

Solution path found is S A G, cost 18

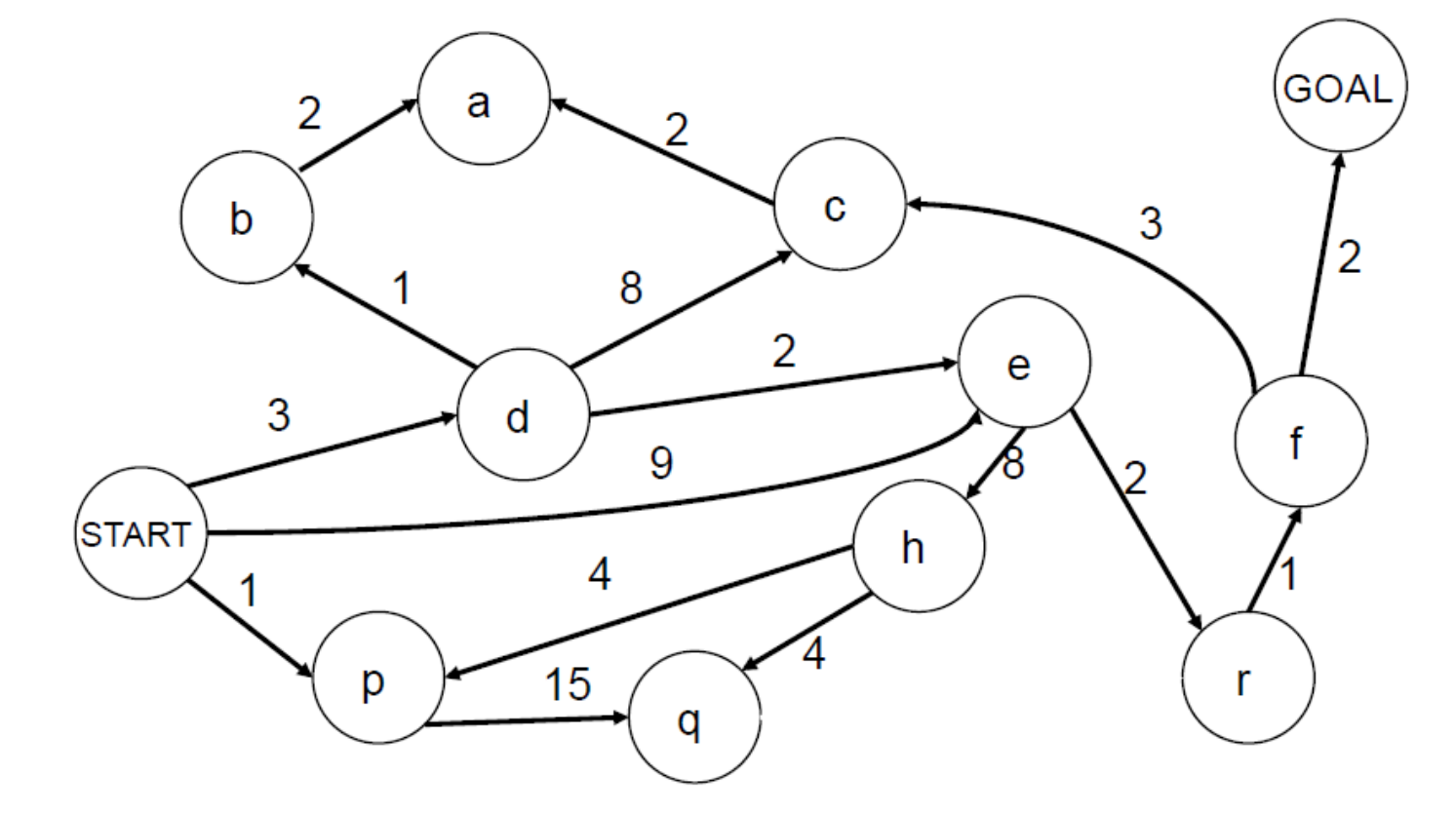
Uniform-Cost Search



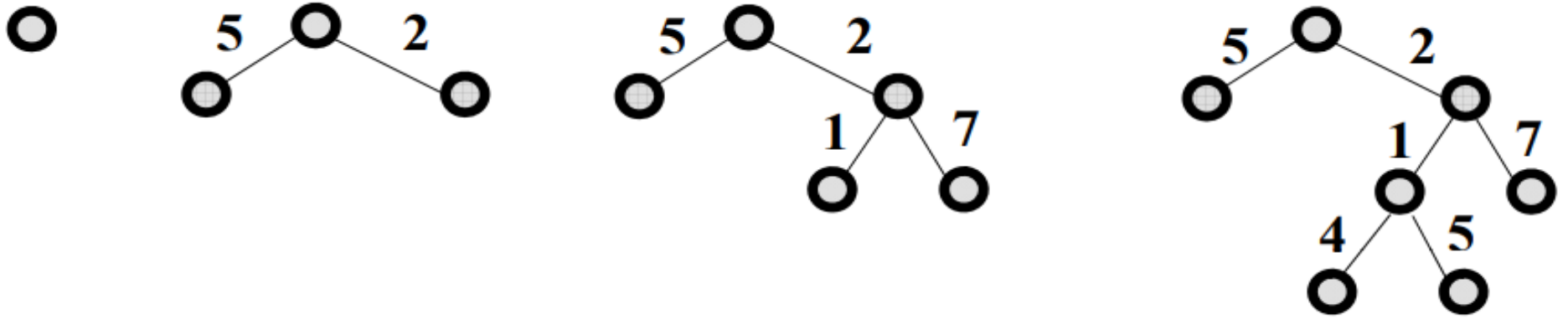
Expanded node	Nodes list
	{ S^0 }
S^0	{ B^1 A^3 C^8 }
B^1	{ A^3 C^8 G^{21} }
A^3	{ D^6 C^8 E^{10} G^{18} G^{21} }
D^6	{ C^8 E^{10} G^{18} G^{21} }
C^8	{ E^{10} G^{13} G^{18} G^{21} }
E^{10}	{ G^{13} G^{18} G^{21} }
G^{13}	{ G^{18} G^{21} }

Solution path found is S C G, cost 13

Exercise



Uniform-Cost Search



- Complete? Yes.
- Optimal? Yes, if path cost is nondecreasing function of depth
- Time Complexity: $O(b^d)$
- Space Complexity: $O(b^d)$, note that every node in the fringe keep in the queue.

When to use What

- **Depth-First Search:**
  - Many solutions exist
  - Know (or have a good estimate of) the depth of solution
- **Breadth-First Search:**
  - Some solutions are known to be shallow
- **Uniform-Cost Search:**
  - Actions have varying costs
  - Least cost solution is required
  - *This is the only uninformed search that worries about costs*.
- **Iterative-Deepening Search:**
  - Space is limited and the shortest solution path is required

Bi-directional Search

- Alternate searching from the start state toward the goal and from the goal state toward the start.
- Stop when the frontiers intersect.
- Works well only when there are unique start and goal states and when actions are reversible.
- Can lead to finding a solution more quickly.

